École Nationale Supérieure d'Arts et Métiers

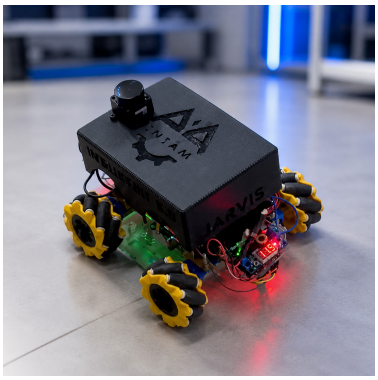
Département AEEE — Génie électromécanique / Digitalisation industrielle

---

Communication cobotique d'un AGV intelligent

HoloLens, jumeau numérique AASX Blazor, commande vocale et intégration Jetson–STM32

Architecture de communication cobotique immersive pour AGV intelligent sur NVIDIA Jetson



Réalisé par :

Rhilan Amine

Imad Msaadi

Encadré par :

M. Sabor Jalal

Année universitaire : 2025–2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à **Sabor Jalal** pour son encadrement, sa disponibilité et ses orientations techniques tout au long des phases de conception, d'intégration et de validation du robot.

Nous remercions également **École Nationale Supérieure d'Arts et Métiers** pour les moyens pédagogiques, techniques et documentaires mis à notre disposition. Ce cadre nous a permis de développer une solution complète combinant mécatronique, informatique embarquée et communication robotique.

Enfin, nous adressons nos remerciements aux membres de l'équipe projet pour leur collaboration lors des essais matériels, de la mise au point du dashboard web, de l'intégration HoloLens/AASX Blazor, de la communication Jetson–STM32 et du voice agent.

Résumé

Synthèse du projet

Ce rapport présente la conception et la validation d'une architecture de **communication cobotique** pour un robot AGV à roues mécanum. L'objectif est de permettre une interaction claire entre l'opérateur et le robot à travers un casque **HoloLens**, un dashboard web, un **jumeau numérique AASX Blazor**, un voice agent intelligent, une supervision temps réel et une communication embarquée fiable entre la NVIDIA Jetson et la STM32.

Le système développé repose sur une architecture distribuée : la **NVIDIA Jetson** assure l'intelligence embarquée, les services réseau, la vision YOLO et la supervision ; la **STM32** exécute les commandes temps réel des moteurs ; les interfaces HoloLens, AASX Blazor, dashboard web et voice agent permettent à l'utilisateur de commander manuellement le robot, d'observer ses mouvements dans une scène 3D d'usine et de lancer des missions par la voix. Les protocoles HTTP, WebRTC, MQTT, ROS 2, SignalR et UART assurent l'échange des ordres, des états et des données de capteurs.

Mots-clés : communication cobotique, AGV, roues mécanum, NVIDIA Jetson, STM32, HoloLens, AASX Blazor, dashboard web, voice agent, MQTT, ROS 2, LiDAR, caméra, YOLO, Flask, WebRTC, jumeau numérique.

Abstract

This report presents the design and validation of a **cobotic communication architecture** for a mecanum-wheeled Automated Guided Vehicle. The proposed system enables clear interaction between the operator and the robot through a HoloLens headset, a web dashboard, an AASX Blazor digital twin, an intelligent voice assistant, real-time supervision and reliable embedded communication between NVIDIA Jetson and STM32.

The architecture combines high-level services running on the NVIDIA Jetson, real-time motor control on STM32, YOLO camera perception, and communication layers based on HTTP, WebRTC, MQTT, ROS 2, SignalR and UART. The result is a supervised mobile robot platform that can be controlled manually, monitored in a 3D factory scene, and extended for voice-based mission execution and human–robot interaction.

Keywords : cobotic communication, AGV, mecanum wheels, NVIDIA Jetson, STM32, HoloLens, AASX Blazor, web dashboard, voice assistant, MQTT, ROS 2, LiDAR, camera, YOLO, Flask, WebRTC, digital twin.

Table des matières

Remerciements i

Résumé ii

Abstract iii

1 Introduction générale 1

1.1 Contexte du projet . . . 1
1.2 Notion de communication cobotique . . . 2
1.3 Problématique . . . 2
1.4 Objectifs du projet . . . 3
1.5 Périmètre du projet . . . 4
1.6 Méthodologie adoptée . . . 4
1.7 Organisation du rapport . . . 4

2 Conception matérielle et firmware STM32 6

2.1 Conception globale du robot AGV . . . 6
2.1.1 Objectif de la conception . . . 6
2.1.2 Architecture fonctionnelle du robot . . . 7
2.1.3 Choix d'une base à roues mécanum . . . 8
2.1.4 Implantation des composants . . . 8
2.1.5 Chaîne d'énergie . . . 9
2.1.6 Chaîne d'information . . . 9
2.1.7 Bilan de conception . . . 10
2.2 Partie hardware et matériel utilisé . . . 10
2.2.1 Objectif de la partie matérielle . . . 10
2.2.2 Matériel principal utilisé . . . 10
2.2.3 Architecture hardware globale . . . 12
2.2.4 NVIDIA Jetson . . . 13
2.2.5 STM32 . . . 14
2.2.6 Drivers, moteurs et encodeurs . . . 14
2.2.7 LiDAR et caméra . . . 16
2.2.8 Alimentation électrique . . . 17
2.3 Firmware STM32 . . . 17

2.3.1 Rôle du firmware . . . 17
2.3.2 Architecture fonctionnelle du firmware . . . 17
2.3.3 Cinématique d'une base mécanum . . . 18
2.3.4 Protocole de communication UART . . . 19
2.3.5 Commande PID des moteurs . . . 20
2.3.6 Gestion du sens de rotation . . . 20
2.3.7 Pseudo-code du firmware . . . 21
2.3.8 Sécurité logicielle . . . 22
2.3.9 Paramètres à calibrer . . . 22

3 Architecture logicielle et communication cobotique 24
3.1 Communication cobotique entre les éléments du système . . . 24
3.1.1 Objectif de la communication . . . 24
3.1.2 Schéma global de communication . . . 25
3.1.3 Lecture du schéma par couches . . . 25
3.1.4 Scénario opérateur augmenté avec HoloLens et AASX Blazor . . . 27
3.1.5 Rôle de chaque protocole . . . 28
3.1.6 Communication Jetson–STM32 . . . 30
3.1.7 Communication MQTT et Node-RED . . . 30
3.1.8 Exemple complet : commande « avance » . . . 32
3.1.9 Bilan . . . 32
3.2 Voice agent basé sur Gemini API . . . 32
3.2.1 Objectif du voice agent . . . 32
3.2.2 Principe général du voice agent . . . 33
3.2.3 Rôle de la clé API Gemini . . . 33
3.2.4 Lien entre le voice agent et la détection caméra YOLO . . . 34
3.2.5 Séquence de traitement d'une commande vocale . . . 35
3.2.6 Conversion des phrases en commandes . . . 36
3.2.7 Sécurité du voice agent . . . 36
3.3 Navigation autonome, simulation Gazebo et jumeau numérique Blazor . . . 36
3.3.1 Objectif de la couche navigation et simulation . . . 36
3.3.2 Architecture ROS 2 pour la navigation autonome . . . 37
3.3.3 Simulation Gazebo et validation locale . . . 38
3.3.4 Envoi des données vers l'interface de supervision . . . 39
3.3.5 Jumeau numérique AASX ASP.NET Blazor . . . 41
3.3.6 Interface 3D de visualisation des mouvements . . . 45
3.3.7 Commande de navigation depuis le jumeau numérique . . . 46
3.3.8 Bilan . . . 47

4 Positionnement du projet selon le modèle RAMI 4.0 48

4.1 Pourquoi utiliser RAMI 4.0 ? . . . 48
4.2 Architecture du projet selon RAMI 4.0 . . . 49
4.3 Lecture par couches . . . 49
4.3.1 Couche 1 – Asset . . . 49
4.3.2 Couche 2 – Intégration . . . 50
4.3.3 Couche 3 – Communication . . . 51
4.3.4 Couche 4 – Information . . . 51
4.3.5 Couche 5 – Fonctionnelle . . . 52
4.3.6 Couche 6 – Business . . . 53
4.4 Lecture selon les niveaux hiérarchiques . . . 54
4.5 Lecture selon le cycle de vie et la chaîne de valeur . . . 54
4.6 Table de correspondance RAMI 4.0 / éléments du projet . . . 55
4.7 Discussion . . . 56

5 Tests et validation 57
5.1 Objectif des essais . . . 57
5.2 Plan de test . . . 57
5.3 Tests sous RViz avant la mise en mouvement du robot réel . . . 59
5.3.1 Objectifs du test RViz . . . 59
5.3.2 Procédure recommandée . . . 59
5.3.3 Checklist de validation sous RViz . . . 60
5.3.4 Interprétation des résultats . . . 61
5.4 Validation de la communication série . . . 61
5.5 Validation de HoloLens et du jumeau numérique AASX Blazor . . . 62
5.6 Validation du dashboard . . . 62
5.7 Validation des mouvements . . . 63
5.8 Problèmes rencontrés et actions correctives . . . 64
5.9 Conclusion de validation . . . 64
5.10 Validation du lien voice agent – YOLO . . . 64
5.11 Validation spécifique du voice agent . . . 65

6 Conclusion générale 66

Bibliographie 67

Table des figures

1.1 Véhicule à guidage automatique utilisé comme plateforme expérimentale . 1
1.2 Principe général de la communication cobotique du système AGV 2

2.1 Architecture materiel du robot 7
2.2 Découpage fonctionnel du robot AGV 7
2.3 Mouvements principaux d'une plateforme à roues mécanum 8
2.4 Organisation physique proposée des composants 9
2.5 Chaîne d'énergie du robot 9
2.6 Chaîne d'information et de commande 10
2.7 Intégration HoloLens, AASX Blazor et Jetson dans l'architecture globale . 12
2.8 NVIDIA Jetson 13
2.9 STM32 14
2.10 L298N double pont H 14
2.11 12V DC Moteur avec Encodeur et réducteur de vitesse 110RPM 400RPM . 15
2.12 Boucle de commande d'un moteur avec encodeur 15
2.13 LiDAR 16
2.14 caméra 16
2.15 Distribution recommandée de l'alimentation 17
2.16 Organisation fonctionnelle du firmware STM32 18

3.1 Schéma global de la chaîne de communication entre l'utilisateur, Flask, le voice agent, MQTT, Node-RED, ROS 2 et le robot AGV 25
3.2 Vue opérateur dans HoloLens : dashboard, commandes et jumeau numérique AASX Blazor 27
3.3 Chaîne d'interaction HoloLens, dashboard, jumeau numérique et voice agent 28
3.4 Échanges série entre Jetson et STM32 30
3.5 Séquence de communication pour une commande d'avancement 32
3.6 Architecture du voice agent connecté à Gemini API et à la Jetson 33
3.7 Dashboard JARVIS : assistant vocal, capteurs et données JSON utilisées pour la supervision 34
3.8 Flux caméra intégré au dashboard et exploité par le voice agent pour décrire l'environnement 34
3.9 Traitement complet d'une commande vocale depuis HoloLens ou navigateur 35

3.10 Chaîne ROS 2 de la navigation autonome 37
3.11 Architecture de simulation Gazebo et diffusion vers les interfaces 38
3.12 Pipeline d'envoi des données ROS 2 vers les interfaces utilisateur 40
3.13 Visualisation du scan LiDAR dans le dashboard : points organisés sous forme de carré représentant les obstacles détectés 40
3.14 Modèle AASX du robot AGV : organisation des submodels Identification, Motion, Navigation, Battery, Sensors, Safety, Missions et Commands . . . 42
3.15 Affichage des submodels et des informations statiques dans le dashboard AASX Blazor 42
3.16 Scène 3D de l'usine dans le jumeau numérique AASX Blazor 43
3.17 Vue rapprochée du modèle AGV JARVIS dans la scène 3D du jumeau numérique 43
3.18 Architecture du jumeau numérique Blazor connecté au robot et à Gazebo . 43
3.19 la scène 3D Three.j 45
3.20 Commande manuelle du robot, missions et canal vocal WebSocket depuis le dashboard affiché dans HoloLens 46

4.1 Architecture du projet AGV JARVIS positionnée selon les axes du modèle RAMI 4.0 49

5.1 Test de l'affichage du dashboard avec jumeau numérique, flux caméra et données de supervision 62

Liste des tableaux

2.1 Liste du matériel utilisé dans le robot AGV 10
2.2 Exemple de protocole série 20
2.3 Paramètres firmware à ajuster expérimentalement 22

3.1 Lecture fonctionnelle du schéma global de communication 26
3.2 Protocoles et liaisons utilisés dans l'architecture de communication cobotique 29
3.3 Topics MQTT proposés pour la supervision et les missions AGV 31
3.4 Exemples d'interaction entre la vision YOLO et le voice agent 35
3.5 Exemples d'intentions vocales, réponses et commandes associées 36
3.6 Composants ROS 2 de la navigation autonome 37
3.7 Topics ROS 2 principaux de la navigation 38
3.8 Topics MQTT pour la navigation et le jumeau numérique 41
3.9 Composants principaux du jumeau numérique Blazor 44

4.1 Correspondance entre RAMI 4.0 et les composants du projet AGV JARVIS 55

5.1 Plan de test fonctionnel du système AGV 57
5.2 Checklist de validation RViz avant déplacement du robot 60
5.3 Mouvements de base à valider 63
5.4 Exemples de problèmes d'intégration et corrections 64
5.5 Tests voice agent liés à la vision YOLO 65
5.6 Tests de validation du voice agent Gemini 65

1. Introduction générale

1.1 Contexte du projet

Les véhicules à guidage automatique, appelés AGV (*Automated Guided Vehicles*), occupent une place importante dans les environnements industriels modernes. Ils permettent de réaliser des missions de déplacement, de transport, d'inspection et de supervision avec une intervention humaine réduite. Dans une logique de communication cobotique, l'objectif n'est pas seulement de rendre le robot mobile, mais aussi de le rendre compréhensible, pilotable et supervisable par un opérateur.



FIGURE 1.1 – Véhicule à guidage automatique utilisé comme plateforme expérimentale

Dans le cadre de ce projet, la plateforme AGV est basée sur une **NVIDIA Jetson**. Cette carte embarquée joue le rôle de calculateur principal : elle centralise la supervision, la vision, les communications réseau, le dashboard web, le jumeau numérique AASX Blazor, l'affichage HoloLens et le voice agent. La commande temps réel des moteurs est confiée à une **STM32**, ce qui permet de séparer la couche intelligente de la couche de contrôle bas niveau.

Le robot repose sur une architecture mécatronique complète : base mobile à roues mécanum, moteurs à courant continu, encodeurs, drivers moteurs, carte STM32, LiDAR, caméra USB, dashboard web, communication MQTT/ROS 2, casque HoloLens, jumeau numérique AASX Blazor et voice agent. Cette combinaison permet de construire une solution cohérente orientée **interaction homme–robot**, supervision immersive et mobilité intelligente.

Dans le scénario d'utilisation retenu, l'utilisateur porte un **casque HoloLens**. Il accède au dashboard, observe le jumeau numérique du robot dans une scène 3D d'usine, suit les mouvements du robot en temps réel et peut choisir entre deux modes de commande : le **pilotage manuel** depuis l'interface ou le **pilotage vocal** à travers un voice agent relié à la caméra et à la détection YOLO.

1.2 Notion de communication cobotique

La communication cobotique désigne ici l'ensemble des échanges permettant à un opérateur de collaborer efficacement avec un robot mobile. Elle regroupe trois dimensions principales :

— la **commande** : l'opérateur transmet des ordres via le dashboard, le casque HoloLens, la voix ou une mission Node-RED ;

— la **supervision** : le robot renvoie son état, ses mesures, sa position, ses alertes et sa représentation dans le jumeau numérique 3D ;

— la **sécurité fonctionnelle** : les commandes critiques, comme l'arrêt d'urgence, doivent être traitées rapidement et de manière fiable.

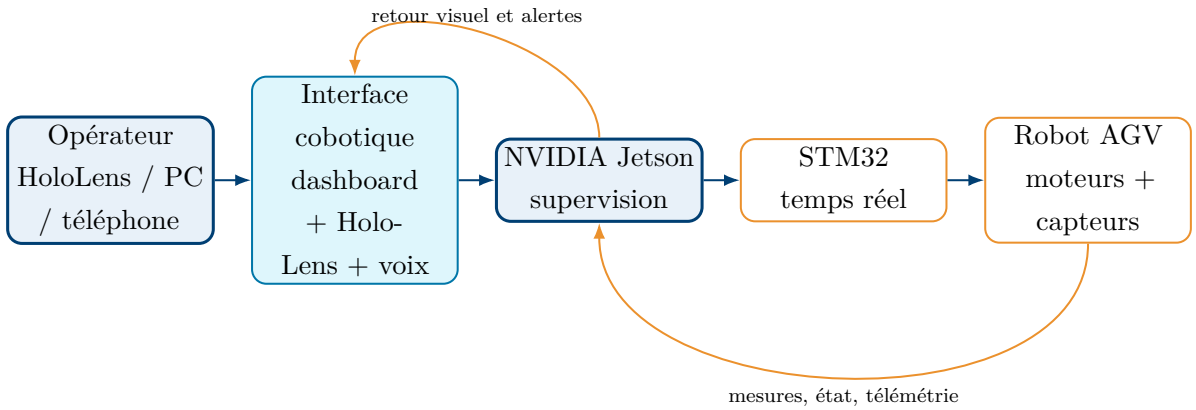


FIGURE 1.2 – Principe général de la communication cobotique du système AGV

1.3 Problématique

La réalisation d'un AGV autonome ou semi-autonome ne se limite pas à la motorisation du robot. Elle exige une intégration cohérente entre plusieurs couches techniques :

— une couche **matérielle** fiable : alimentation, moteurs, capteurs, câblage et protec-

tions ;

— une couche **firmware** déterministe : commande moteurs, PID, encodeurs et sécurité ;

— une couche **logicielle embarquée** : ROS 2, dashboard, MQTT, Node-RED et services réseau ;

— une couche **perception** : caméra, LiDAR et détection par YOLO ;

— une couche **interaction** : HoloLens, dashboard web, jumeau numérique AASX Blazor, voice agent et retour d'état vers l'utilisateur.

Problématique

Comment concevoir une architecture de communication cobotique permettant à un AGV basé sur NVIDIA Jetson d'être piloté depuis un dashboard ou un casque HoloLens, supervisé à travers un jumeau numérique AASX Blazor, commandé par la voix et sécurisé en temps réel, tout en assurant une communication fiable avec la STM32, les capteurs et les actionneurs ?

1.4 Objectifs du projet

L'objectif général est de concevoir une plateforme AGV fonctionnelle, évolutive et documentée, capable d'assurer le déplacement, la supervision et l'interaction avec l'utilisateur.

Les objectifs spécifiques sont les suivants :

1. concevoir une architecture matérielle claire autour de la Jetson, de la STM32, des capteurs et des drivers ;
2. mettre en place un dashboard web permettant le pilotage, la supervision et l'arrêt d'urgence logiciel ;
3. intégrer l'utilisation d'un casque HoloLens pour accéder à l'interface de commande et au jumeau numérique ;
4. assurer la communication entre ROS 2, MQTT, Node-RED et le firmware STM32 ;
5. intégrer la lecture des encodeurs et la commande des moteurs par PID ;
6. afficher les flux caméra, les données LiDAR et les informations de télémétrie ;
7. intégrer une détection par YOLOv8 afin d'identifier les objets ou personnes visibles par la caméra ;
8. ajouter un voice agent capable de transformer un ordre naturel en commande robot

et d'expliquer à l'utilisateur ce que la caméra détecte ;

9. préparer l'extension vers la navigation autonome avec ROS 2, Nav2 et SLAM ;
10. valider les scénarios de communication sur robot réel et en simulation ;
11. développer un jumeau numérique AASX ASP.NET Blazor avec visualisation 3D des mouvements du robot dans une scène d'usine.

1.5 Périmètre du projet

Le projet couvre la conception, l'intégration et les essais fonctionnels du système. Il ne vise pas à certifier le robot pour un usage industriel final, mais à établir une base technique robuste pour des développements futurs : navigation autonome, cartographie, évitement d'obstacles, gestion de missions et amélioration de l'interaction homme–robot.

1.6 Méthodologie adoptée

La méthodologie suivie repose sur une intégration progressive :

1. validation séparée des composants matériels ;
2. test du firmware STM32 avec moteurs et encodeurs ;
3. mise en place des communications série et réseau ;
4. développement du dashboard Flask et préparation de son affichage dans HoloLens ;
5. intégration de MQTT, ROS 2 et rosbridge ;
6. ajout du LiDAR, de la caméra et de YOLO ;
7. intégration de la commande vocale, de la conversion d'intentions et du lien avec les détections YOLO ;
8. développement du jumeau numérique AASX Blazor et de l'interface 3D représentant le robot dans l'usine ;
9. tests complets sur robot réel et en simulation.

1.7 Organisation du rapport

Ce rapport est organisé en plusieurs chapitres complémentaires :

— le **chapitre 1** présente le contexte, la problématique, les objectifs et la méthodologie ;

— le **chapitre 2** regroupe la conception globale, le hardware et le firmware STM32 ;

— le **chapitre 3** décrit la communication cobotique, le voice agent, la navigation ROS 2, la simulation Gazebo et le jumeau numérique AASX Blazor ;

— le **chapitre 4** positionne le projet selon le modèle RAMI 4.0 et justifie la cohérence de son architecture Industrie 4.0 ;

— le **chapitre 5** présente les tests, la validation et la procédure de vérification sous RViz avant déplacement réel ;

— une **conclusion générale** synthétise les principaux apports du projet ;

— une **bibliographie** regroupe les références techniques utilisées.

2. Conception matérielle et firmware STM32

2.1 Conception globale du robot AGV

2.1.1 Objectif de la conception

La conception du robot a pour objectif d'obtenir une plateforme mobile stable, compacte et facilement évolutive. Le robot doit pouvoir se déplacer dans toutes les directions grâce à une base à roues mécanum, tout en embarquant l'ensemble des éléments nécessaires : alimentation, commande temps réel, calcul haut niveau, capteurs, caméra, LiDAR et interface de communication.

Le principe retenu consiste à séparer clairement les fonctions : la **NVIDIA Jetson** assure l'intelligence embarquée et la supervision, tandis que la **STM32** assure la commande temps réel des moteurs.

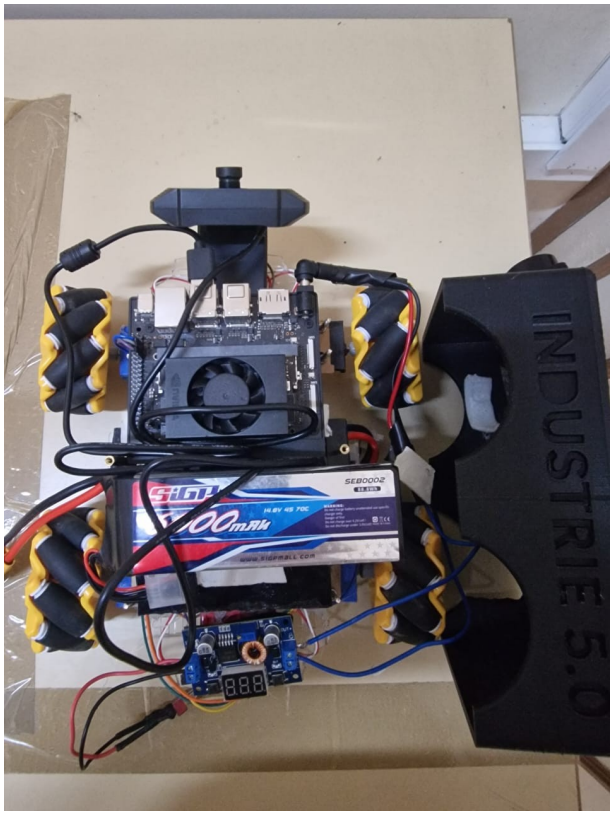


FIGURE 2.1 – Architecture materiel du robot

2.1.2 Architecture fonctionnelle du robot

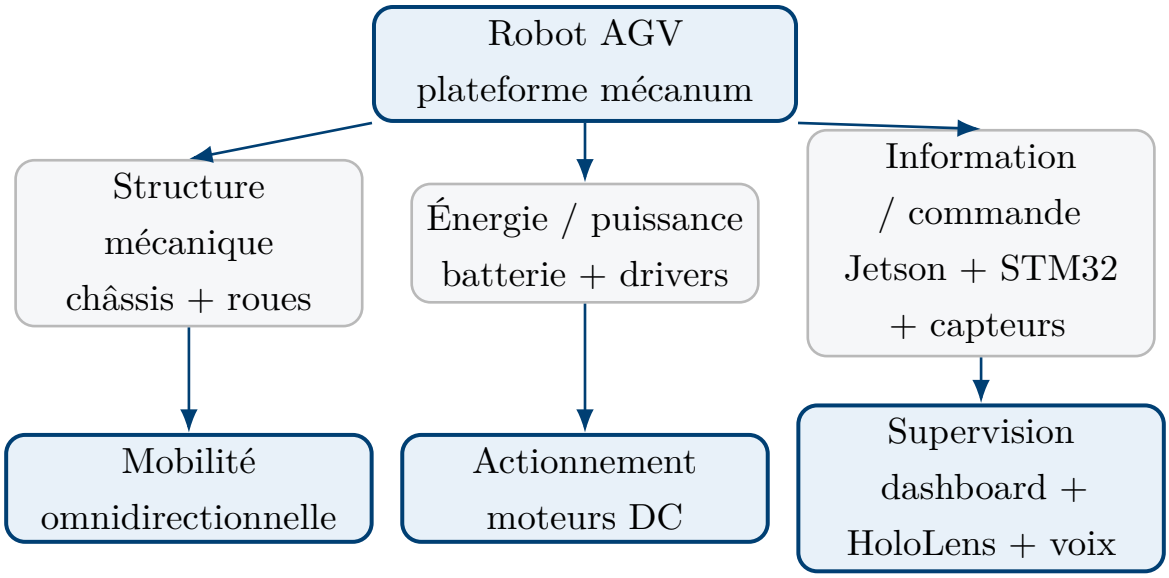


FIGURE 2.2 – Découpage fonctionnel du robot AGV

2.1.3 Choix d'une base à roues mécanum

Le robot utilise quatre roues mécanum. Ce choix permet d'obtenir une mobilité plus riche qu'une base différentielle classique. En commandant séparément les quatre roues, le robot peut avancer, reculer, se déplacer latéralement, tourner sur place ou combiner translation et rotation.

Intérêt des roues mécanum

Les roues mécanum sont adaptées aux robots de démonstration, de transport léger et d'inspection, car elles permettent de corriger rapidement la position du robot sans changer son orientation.

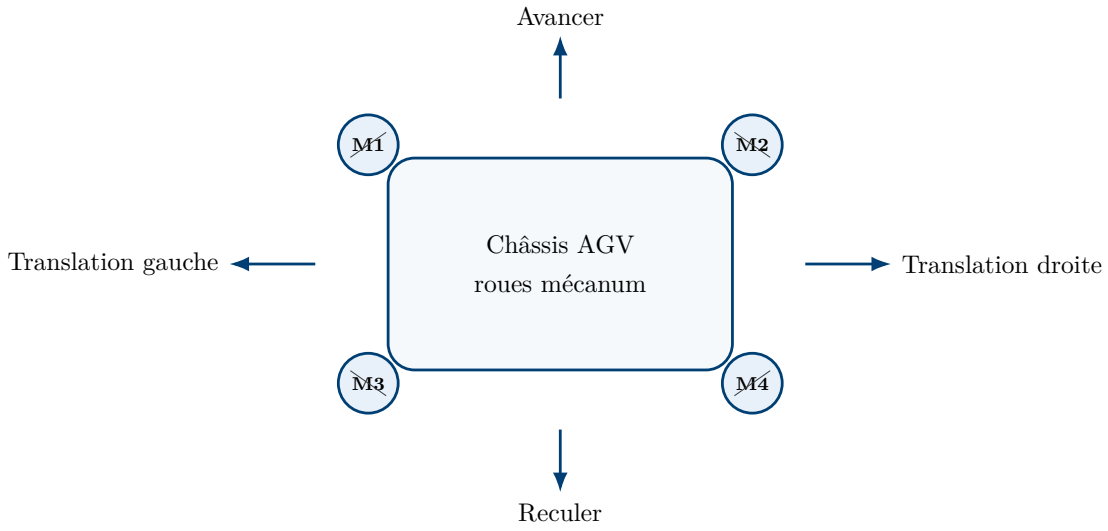


FIGURE 2.3 – Mouvements principaux d'une plateforme à roues mécanum

2.1.4 Implantation des composants

L'implantation doit respecter deux contraintes : l'équilibre mécanique et la réduction des perturbations électriques. La batterie est placée près du centre pour équilibrer le robot. Les drivers sont proches des moteurs afin de limiter la longueur des câbles de puissance. La STM32 est placée proche des drivers et des encodeurs. La Jetson doit rester ventilée et accessible pour les connexions USB, Ethernet ou Wi-Fi.

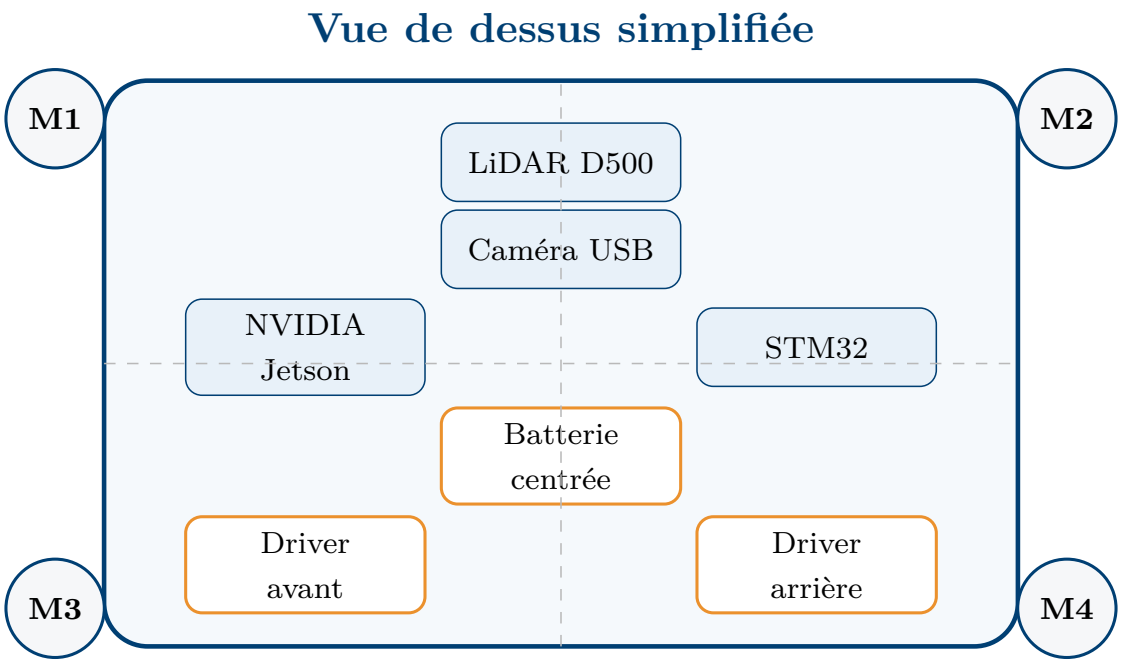


FIGURE 2.4 – Organisation physique proposée des composants

2.1.5 Chaîne d'énergie

La chaîne d'énergie décrit le chemin de l'énergie électrique depuis la batterie jusqu'aux moteurs. Elle comprend la batterie, les protections, les convertisseurs DC-DC, les drivers moteurs et les moteurs DC.

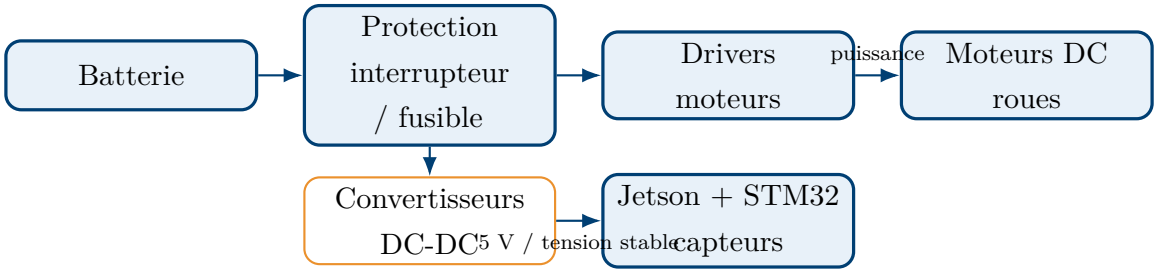


FIGURE 2.5 – Chaîne d'énergie du robot

2.1.6 Chaîne d'information

La chaîne d'information décrit la circulation des données : acquisition par les capteurs, traitement par la Jetson, commande par la STM32 et retour de télémétrie vers l'utilisateur.

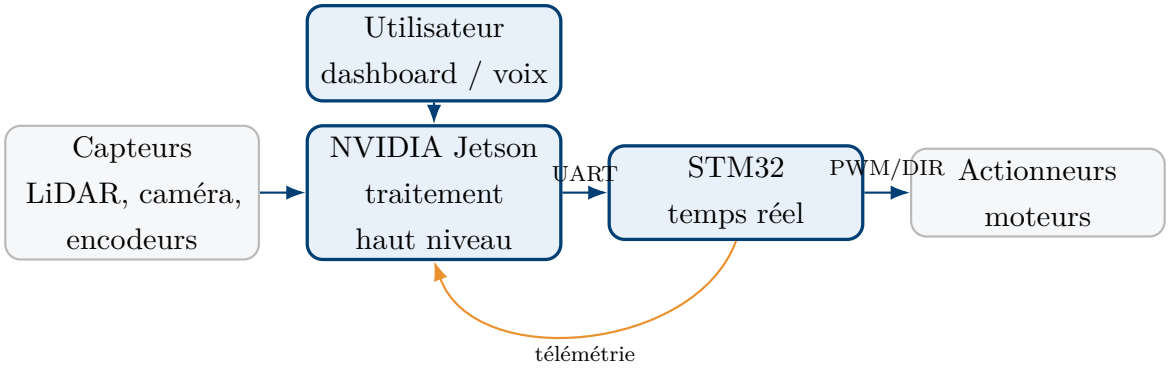


FIGURE 2.6 – Chaîne d'information et de commande

2.1.7 Bilan de conception

Résultat attendu

La conception retenue sépare clairement les rôles : la Jetson gère la supervision, la vision et la commande intelligente ; la STM32 gère la commande moteur temps réel ; les drivers assurent la puissance ; les capteurs alimentent la décision et la supervision.

2.2 Partie hardware et matériel utilisé

2.2.1 Objectif de la partie matérielle

La partie hardware constitue la base physique du robot AGV. Elle regroupe la structure mécanique, la motorisation, l'alimentation, les cartes électroniques, les capteurs et les interfaces de communication. Une bonne organisation matérielle est essentielle pour obtenir un robot fiable, stable et facile à maintenir.

2.2.2 Matériel principal utilisé

TABLE 2.1 – Liste du matériel utilisé dans le robot AGV

Élément	Rôle	Remarque d'intégration
NVIDIA Jetson	Calculateur principal	Exécute Flask, ROS 2, MQTT, YOLO, rosbridge et le voice agent

Élément	**Rôle**	**Remarque d'intégration**
STM32	Commande temps réel	Génère les PWM, lit les encodeurs et applique le PID
Châssis AGV	Support mécanique	Porte les moteurs, batteries, cartes et capteurs
Quatre roues mécanum	Mobilité omnidirectionnelle	Permettent les déplacements longitudinal, latéral et rotation
Quatre moteurs DC	Actionnement	Chaque moteur entraîne une roue
Encodeurs	Mesure de vitesse	Utilisés pour la boucle de régulation
Drivers moteurs	Interface de puissance	Reçoivent PWM et direction depuis la STM32
LiDAR D500	Perception d'obstacles	Fournit des distances autour du robot
Caméra USB	Vision	Retour vidéo et détection YOLO
Batterie	Source d'énergie	Alimente la puissance et les convertisseurs
Convertisseurs DC-DC	Adaptation de tension	Alimentent séparément la logique et la Jetson
Bouton d'arrêt d'urgence	Sécurité	Doit couper la commande ou la puissance moteur

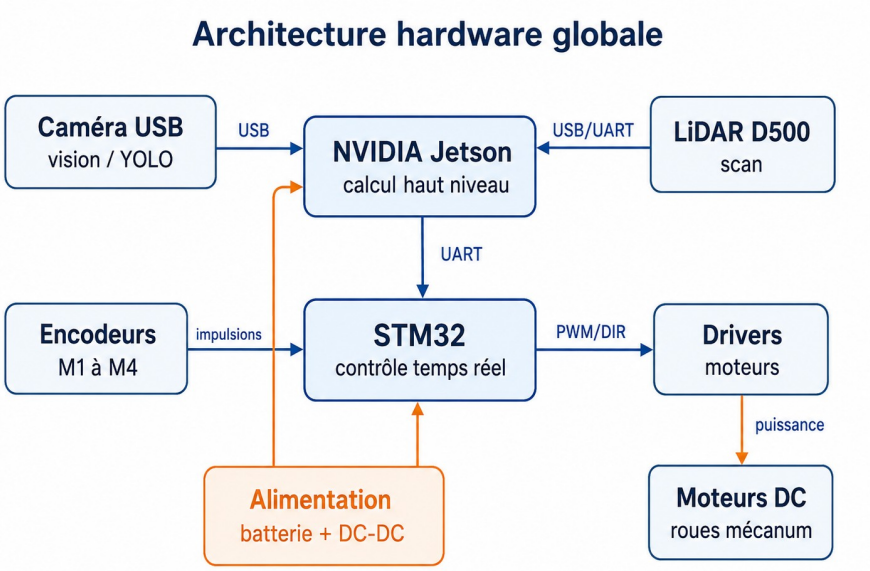


Figure 2.7 – Architecture matérielle globale du robot

2.2.3 Architecture hardware globale

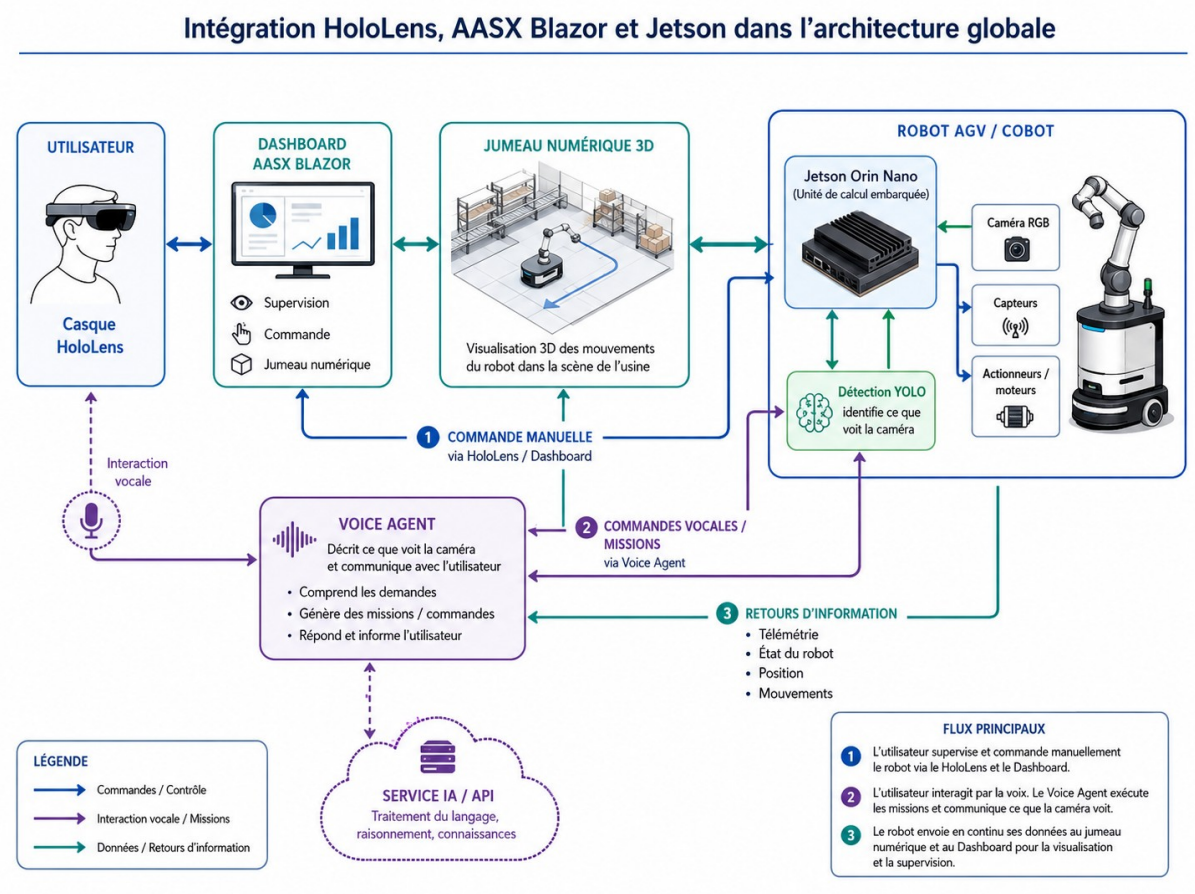


FIGURE 2.7 – Intégration HoloLens, AASX Blazor et Jetson dans l'architecture globale

2.2.4 NVIDIA Jetson



FIGURE 2.8 – NVIDIA Jetson

La NVIDIA Jetson est le calculateur haut niveau du système. Elle reçoit les données capteurs, héberge les services logiciels et transmet les consignes à la STM32. Elle est utilisée pour :

— le dashboard Flask ;

— interface HoloLens et jumeau numérique AASX Blazor ;

— ROS 2 et rosbridge ;

— MQTT et Node-RED ;

— le traitement caméra et YOLO ;

— le voice agent connecté à Gemini API ;

— la communication UART avec la STM32.

2.2.5 STM32

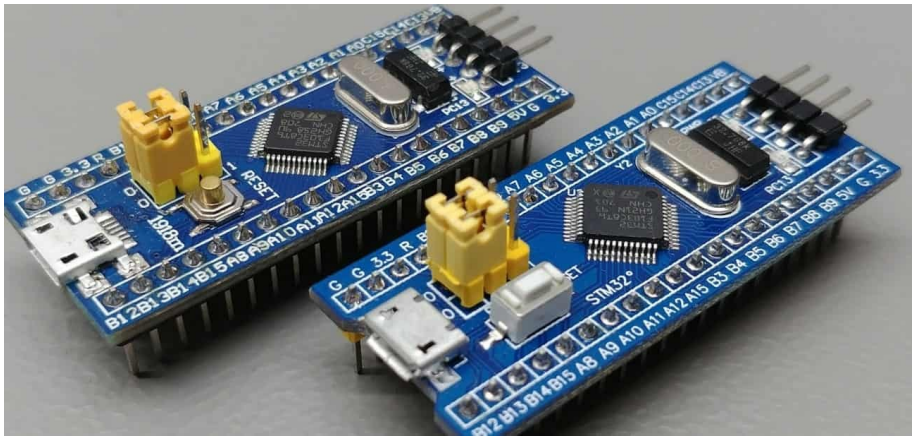


FIGURE 2.9 – STM32

La STM32 est chargée des tâches qui nécessitent une réponse rapide et régulière. Elle est utilisée comme carte de commande bas niveau. Elle lit les encodeurs, calcule les vitesses, applique la régulation PID et génère les signaux PWM vers les drivers moteurs.

Séparation Jetson–STM32

La Jetson décide et supervise. La STM32 exécute la commande moteur. Cette séparation rend le robot plus fiable, car la commande temps réel reste indépendante des charges logicielles de Linux.

2.2.6 Drivers, moteurs et encodeurs

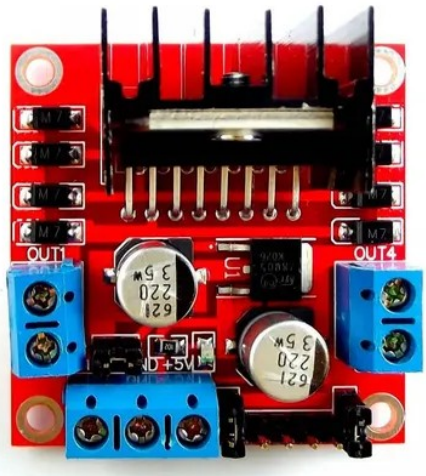


FIGURE 2.10 – L298N double pont H



FIGURE 2.11 – 12V DC Moteur avec Encodeur et réducteur de vitesse 110RPM 400RPM

Les moteurs ne sont pas alimentés directement par la STM32. La STM32 fournit uniquement des signaux de commande. Les drivers moteurs reçoivent ces signaux et fournissent le courant nécessaire aux moteurs.

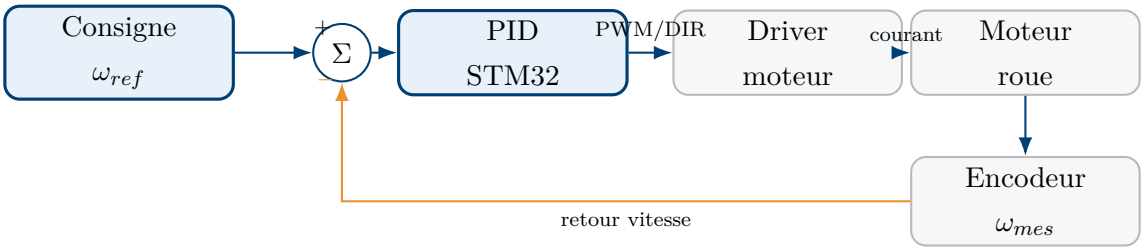


FIGURE 2.12 – Boucle de commande d'un moteur avec encodeur

2.2.7 LiDAR et caméra



FIGURE 2.13 – LiDAR



FIGURE 2.14 – caméra

Le LiDAR D500 permet de mesurer l'environnement autour du robot. Il peut être exploité par ROS 2 sous forme de données de scan. La caméra USB fournit un retour vidéo dans le dashboard et sert aussi à la détection de personnes avec YOLO.

2.2.8 Alimentation électrique

L'alimentation doit être organisée en séparant la puissance moteur et la partie logique. Les moteurs peuvent provoquer des appels de courant importants ; il est donc préférable d'alimenter la Jetson et la STM32 via des convertisseurs DC-DC stables.

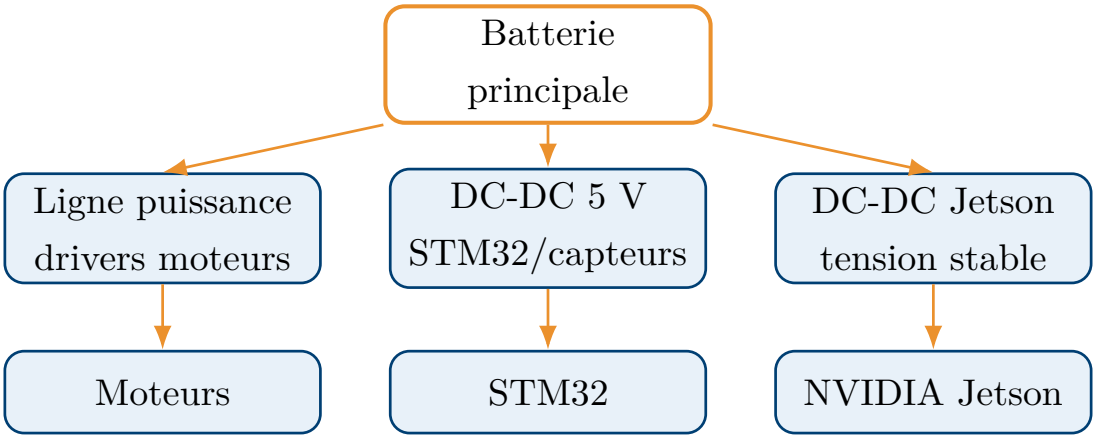


FIGURE 2.15 – Distribution recommandée de l'alimentation

Règles de câblage

Les masses de la Jetson, de la STM32 et des drivers doivent être communes. Les câbles moteurs doivent être séparés des câbles de signaux faibles afin de réduire les perturbations.

2.3 Firmware STM32

2.3.1 Rôle du firmware

Le firmware STM32 constitue la couche bas niveau du système AGV. Son rôle est d'assurer la commande temps réel des moteurs, la lecture des encodeurs, la régulation de vitesse et la communication avec la NVIDIA Jetson.

Contrairement aux applications exécutées sur la Jetson, le firmware est conçu pour être déterministe. Il exécute les tâches critiques avec une période fixe, généralement autour de 20 ms, soit une fréquence de contrôle de 50 Hz.

2.3.2 Architecture fonctionnelle du firmware

Le firmware est organisé en plusieurs blocs :

— initialisation des horloges, GPIO, UART, timers et PWM ;

— réception des commandes série depuis la Jetson ;

— décodage du protocole de communication ;

— calcul des consignes de roues ;

— lecture des encodeurs ;

— régulation PID de vitesse ;

— génération PWM vers les drivers ;

— émission de la télémétrie ;

— gestion des sécurités.

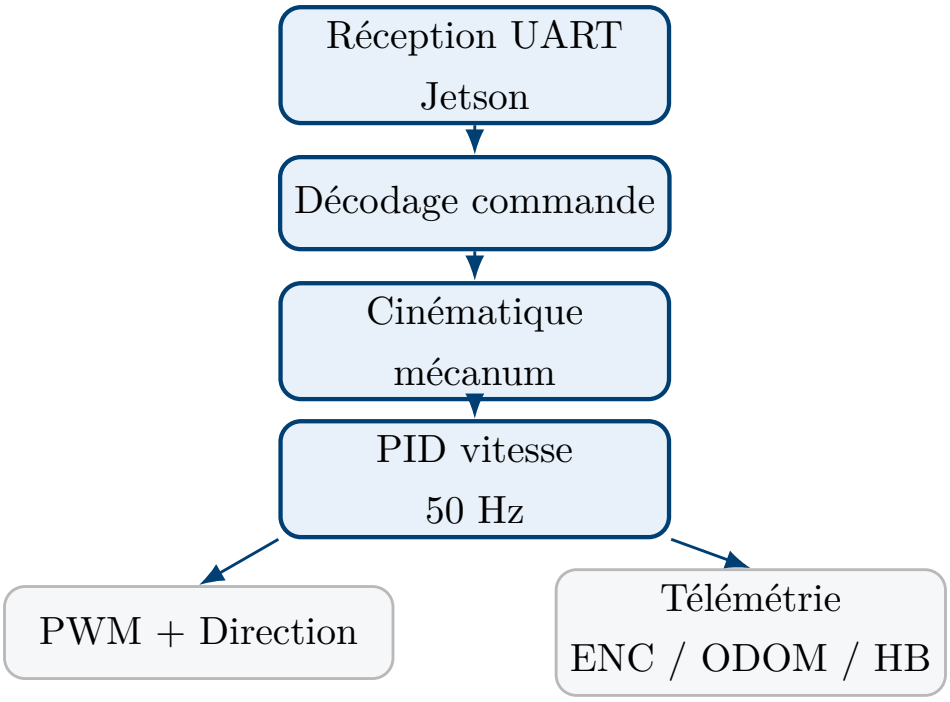


FIGURE 2.16 – Organisation fonctionnelle du firmware STM32

2.3.3 Cinématique d'une base mécanum

Pour une plateforme à quatre roues mécanum, la consigne globale du robot peut être représentée par :

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} \tag{2.1}$$

où v_x est la vitesse longitudinale, v_y la vitesse latérale et ω_z la vitesse angulaire de rotation du robot.

Les vitesses des roues peuvent être calculées par :

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -(L+l) \\ 1 & 1 & (L+l) \\ 1 & 1 & -(L+l) \\ 1 & -1 & (L+l) \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} \tag{2.2}$$

avec :

— r : rayon de la roue ;

— L : demi-longueur du châssis ;

— l : demi-largeur du châssis ;

— ω_i : vitesse angulaire de la roue i.

Convention importante

Les signes de la matrice dépendent du sens de montage des roues, du repérage choisi et du câblage des moteurs. Après montage, il faut vérifier expérimentalement les quatre mouvements de base : avant, arrière, gauche/droite et rotation.

2.3.4 Protocole de communication UART

La communication entre Jetson et STM32 peut être réalisée par UART. Le protocole doit être simple, lisible et robuste.

TABLE 2.2 – Exemple de protocole série

Message	Sens	Rôle
`CMD vx vy wz`	Jetson vers STM32	Commande de vitesse globale du robot
`MV w1 w2 w3 w4`	Jetson vers STM32	Commande directe des vitesses roues
`STOP`	Jetson vers STM32	Arrêt immédiat des moteurs
`ENC e1 e2 e3 e4`	STM32 vers Jetson	Valeurs encodeurs
`ODOM x y theta`	STM32 vers Jetson	Estimation odométrique
`HB t`	STM32 vers Jetson	Signal de vie du firmware

2.3.5 Commande PID des moteurs

Chaque moteur est régulé par un correcteur PID. Le correcteur compare la vitesse mesurée par l'encodeur à la consigne de vitesse de la roue.

$$e(t) = \omega_{\text{ref}}(t) - \omega_{\text{mes}}(t) \tag{2.3}$$

$$u(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de(t)}{dt} \tag{2.4}$$

Dans l'implémentation numérique, le PID est calculé à chaque période d'échantillonnage :

$$u[k] = K_p e[k] + K_i T_e \sum_{i=0}^{k} e[i] + K_d \frac{e[k] - e[k-1]}{T_e} \tag{2.5}$$

Le signal $u[k]$ est ensuite limité dans une plage compatible avec le PWM.

2.3.6 Gestion du sens de rotation

Le signe de la consigne détermine le sens de rotation du moteur :

— si la consigne est positive : rotation dans le sens avant ;

— si la consigne est négative : rotation dans le sens arrière ;

— si la consigne est nulle : arrêt ou freinage selon le driver.

Le rapport cyclique PWM est calculé à partir de la valeur absolue de la commande :

$$PWM = \min\left(PWM_{\max}, |u[k]|\right) \tag{2.6}$$

2.3.7 Pseudo-code du firmware

Listing 2.1 – Pseudo-code simplifié du firmware STM32

```
int main(void)
{
  System_Init();
  GPIO_Init();
  UART_Init();
  Timers_PWM_Init();
  Encoders_Init();

  while (1)
  {
    if (UART_Command_Available())
    {
      Parse_Command();
      Update_Robot_Setpoint();
      Reset_Watchdog_Timer();
    }

    if (Control_Period_Elapsed())   // Exemple : 50 Hz
    {
      Read_Encoders();
      Compute_Wheel_Speeds();
      Compute_PID();
      Apply_PWM_To_Motors();
      Send_Telemetry();
    }

    if (Communication_Timeout())
    {
      Stop_All_Motors();
    }
  }
}
```

2.3.8 Sécurité logicielle

La sécurité du firmware est indispensable pour éviter les comportements dangereux du robot. Les mécanismes recommandés sont :

— arrêt automatique si aucune commande n'est reçue pendant un délai défini ;

— limitation des consignes maximales ;

— limitation du PWM maximal ;

— rampe d'accélération pour éviter les à-coups ;

— arrêt immédiat sur commande `STOP` ;

— envoi périodique d'un message de vie `HB`.

2.3.9 Paramètres à calibrer

TABLE 2.3 – Paramètres firmware à ajuster expérimentalement

Paramètre	**Rôle**	**Méthode de calibration**
K_p	Réactivité du moteur	Augmenter progressivement jusqu'à obtenir une réponse rapide
K_i	Correction de l'erreur statique	Ajouter faiblement après stabilisation du K_p
K_d	Amortissement	Utiliser seulement si oscillations importantes
$PWM_{\max}$	Limitation de puissance	Fixer selon les moteurs et drivers
T_e	Période de régulation	Choisir une valeur stable, par exemple 20 ms
N_{ppr}	Résolution encodeur	Dépend de l'encodeur et du réducteur

Validation attendue

Le firmware est considéré comme validé lorsque les quatre roues répondent correctement aux consignes, que les vitesses mesurées suivent les références et que le robot s'arrête automatiquement en cas de perte de communication.

3. Architecture logicielle et communication cobotique

3.1 Communication cobotique entre les éléments du système

3.1.1 Objectif de la communication

L'architecture de communication du système AGV a pour objectif de relier l'utilisateur, l'interface web, le backend, les services d'intelligence, la couche ROS 2 et la chaîne robotique embarquée. Le système ne fonctionne donc pas comme un bloc unique, mais comme une chaîne distribuée dans laquelle chaque élément reçoit, transforme puis redistribue une information utile : commande, télémétrie, flux vidéo, scan LiDAR, état de mission ou retour vocal.

Le principe retenu est le suivant : l'utilisateur interagit avec le **dashboard web** depuis un navigateur ou depuis le casque **HoloLens**. Le serveur **Flask** joue le rôle de point central de supervision : il reçoit les actions de l'interface, normalise les commandes, sert les flux vidéo, récupère les états issus de l'API AASX et échange des données JSON avec le **voice agent**. Les flux robotiques sont ensuite orchestrés par **Node-RED**, **MQTT**, **rosbridge** et **ROS 2**, avant d'être transmis à la chaîne embarquée Jetson–STM32–robot.

Cette organisation permet de séparer clairement quatre types de flux : les **commandes et le contrôle**, les **données et la télémétrie**, les flux **voix / intelligence artificielle** et les échanges de **supervision / interface**. Cette séparation rend le système plus lisible, plus facile à dépanner et plus évolutif.

3.1.2 Schéma global de communication

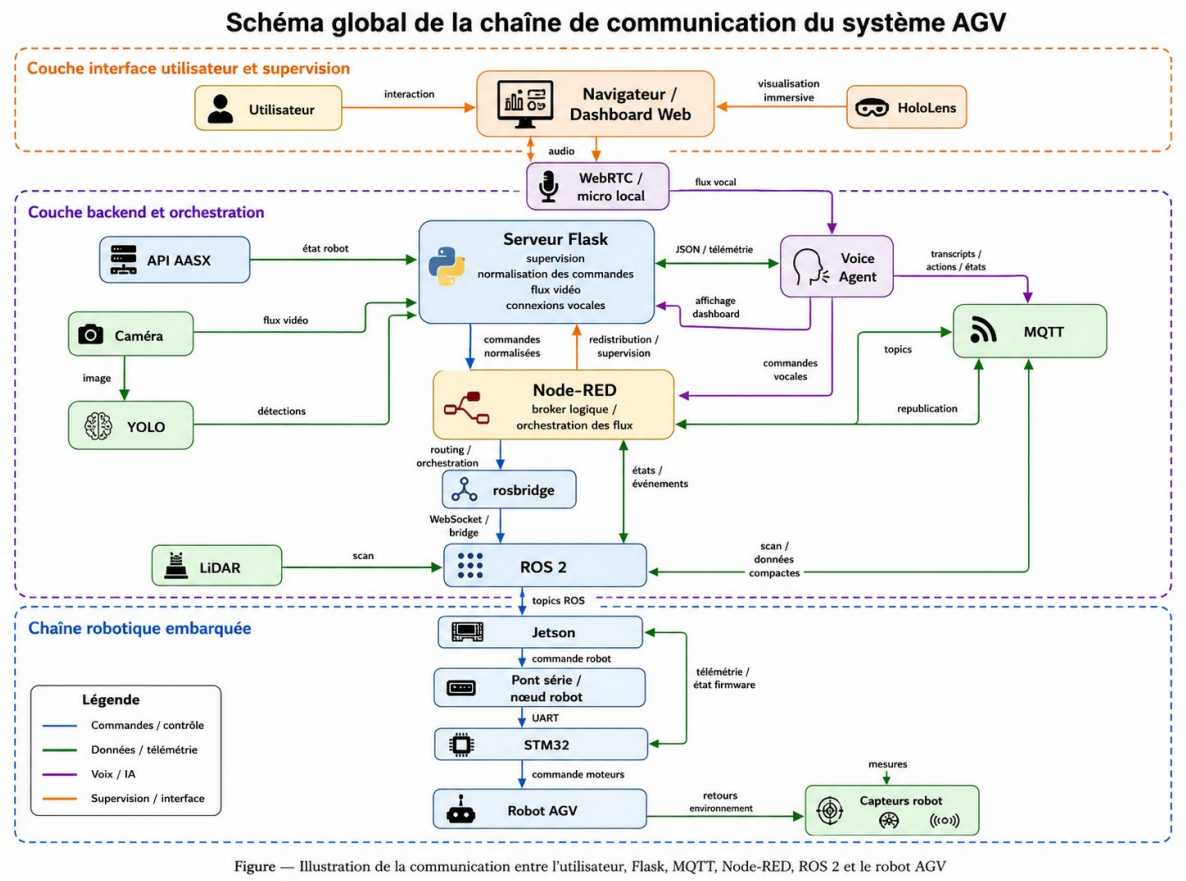


Figure — Illustration de la communication entre l'utilisateur, Flask, MQTT, Node-RED, ROS 2 et le robot AGV

FIGURE 3.1 – Schéma global de la chaîne de communication entre l'utilisateur, Flask, le voice agent, MQTT, Node-RED, ROS 2 et le robot AGV

3.1.3 Lecture du schéma par couches

Le schéma précédent structure la communication du système en trois couches principales. Cette lecture par couches facilite la compréhension du rôle de chaque module et du sens des échanges.

TABLE 3.1 – Lecture fonctionnelle du schéma global de communication

Couche	**Éléments principaux**	**Fonction dans le système**
Interface utilisateur et supervision	Utilisateur, navigateur, dashboard web, HoloLens, WebRTC/micro local	Permettre à l'opérateur de visualiser l'état du robot, d'envoyer des commandes, de suivre le jumeau numérique et de transmettre des ordres vocaux.
Backend et orchestration	Flask, API AASX, caméra, YOLO, voice agent, MQTT, Node-RED, rosbridge, ROS 2, LiDAR	Centraliser l'information, normaliser les commandes, convertir la voix en actions, diffuser les états, router les flux vidéo et synchroniser les données robot.
Chaîne robotique embarquée	ROS 2, Jetson, pont série, STM32, robot AGV, capteurs robot	Exécuter les ordres de mouvement, lire les mesures, transmettre la télémétrie et assurer le lien entre la décision logicielle et l'action physique.

Dans la **couche interface utilisateur**, l'opérateur peut utiliser un navigateur classique ou une visualisation immersive HoloLens. Les commandes manuelles partent du dashboard, tandis que l'audio du microphone est envoyé vers le voice agent à travers WebRTC ou un flux audio local.

Dans la **couche backend et orchestration**, Flask constitue le noyau de supervision. Il reçoit l'état robot depuis l'API AASX, les flux vidéo de la caméra, les détections YOLO et les intentions produites par le voice agent. Node-RED joue le rôle de courtier logique : il organise les scénarios, route les commandes normalisées, déclenche les missions et republie les informations vers MQTT ou ROS 2 selon le besoin.

Dans la **chaîne robotique embarquée**, ROS 2 transmet les informations robotiques sous forme de topics. La Jetson reçoit ces topics, prépare les commandes robot et les envoie vers la STM32 à travers le pont série. La STM32 applique ensuite les commandes moteurs, récupère les retours capteurs et renvoie l'état firmware vers la Jetson et les couches de supervision.

3.1.4 Scénario opérateur augmenté avec HoloLens et AASX Blazor

Dans l'architecture proposée, l'utilisateur peut porter un **casque HoloLens** pour accéder à l'interface de supervision sans rester devant un poste fixe. Le casque affiche le dashboard et le jumeau numérique **AASX Blazor**. L'opérateur peut donc observer le robot réel, suivre son équivalent numérique dans une scène 3D de l'usine et envoyer des commandes sans perdre le contexte visuel de l'environnement.

Le jumeau numérique ne se limite pas à une simple représentation graphique. Il reçoit les états du robot en temps réel : position, orientation, vitesse, mode de fonctionnement, niveau de batterie, état de mission, informations LiDAR et résultats de la détection caméra. Cette synchronisation permet de visualiser les mouvements de l'AGV dans l'usine, même lorsque le robot est partiellement caché ou éloigné de l'opérateur.

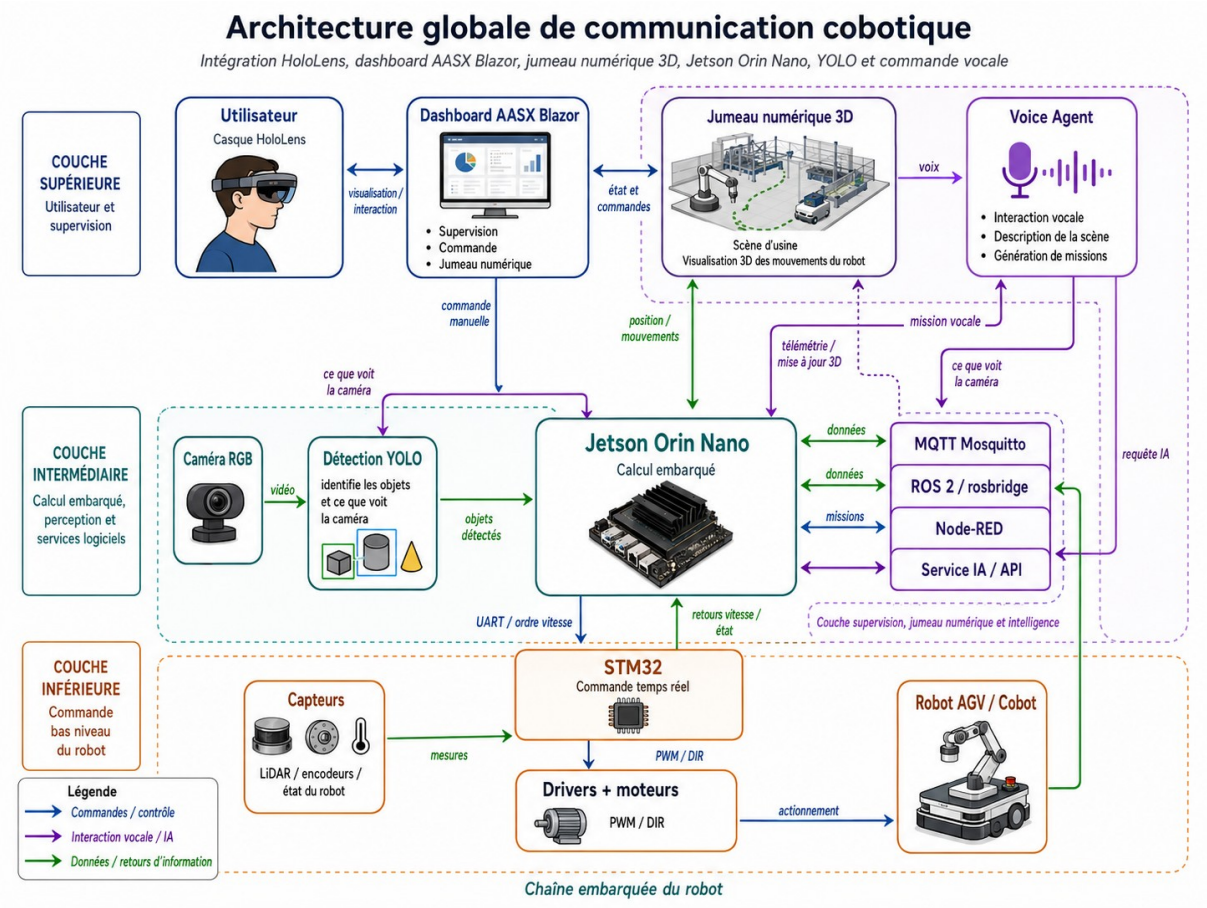


FIGURE 3.2 – Vue opérateur dans HoloLens : dashboard, commandes et jumeau numérique AASX Blazor

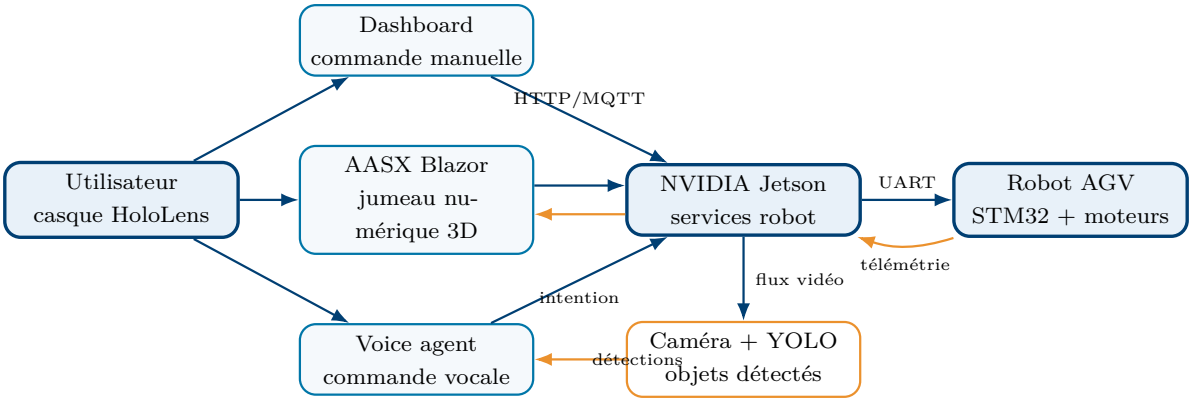


FIGURE 3.3 – Chaîne d'interaction HoloLens, dashboard, jumeau numérique et voice agent

Modes de commande proposés

Deux modes d'utilisation sont prévus :

— **mode manuel** : l'utilisateur commande directement le robot depuis le dashboard affiché dans HoloLens ou depuis un navigateur classique. Les boutons de déplacement génèrent des consignes de vitesse envoyées vers la Jetson ;

— **mode voice agent** : l'utilisateur donne un ordre vocal. Le voice agent analyse la phrase, consulte si nécessaire les résultats YOLO de la caméra, puis transforme la demande en mission ou en commande robot normalisée.

Exemple : si l'utilisateur demande « qu'est-ce que le robot voit ? », la Jetson récupère les classes détectées par YOLO, prépare une réponse compréhensible et l'affiche ou l'annonce à l'utilisateur. Si l'utilisateur dit « lance une mission vers la zone de stockage », le voice agent convertit la phrase en objectif ou en mission MQTT après vérification de sécurité.

3.1.5 Rôle de chaque protocole

Chaque protocole est choisi selon la nature du flux à transporter. Les commandes de supervision doivent être simples et rapides, les flux robot doivent être structurés, et les échanges vocaux doivent rester séparés de la commande bas niveau afin de garantir la sécurité.

TABLE 3.2 – Protocoles et liaisons utilisés dans l'architecture de communication cobotique

Protocole / service	**Liaison**	**Rôle**
HTTP / JSON	Navigateur – Flask	Accès au dashboard, envoi des actions utilisateur et affichage des états sous forme JSON.
WebRTC / audio local	Navigateur ou HoloLens – voice agent	Transport du flux vocal entre le microphone utilisateur et le service d'analyse vocale.
Flask	Dashboard, API AASX, caméra, YOLO, voice agent	Serveur central : supervision, normalisation des commandes, distribution vidéo et échanges de télémétrie.
API AASX	AASX – Flask	Transmission de l'état robot structuré selon une représentation de type jumeau numérique.
Gemini API	Voice agent – service IA	Compréhension de la commande vocale et production d'une intention exploitable.
MQTT	Voice agent, Node-RED, ROS 2, dashboard	Publication/souscription de commandes, événements, états robot et messages de mission.
Node-RED	Flask, MQTT, rosbridge, ROS 2	Orchestration logique des flux, routage des missions et redistribution des informations vers les interfaces.
rosbridge / WebSocket	Node-RED ou dashboard – ROS 2	Passage entre les services web et les topics ROS 2.
ROS 2	Jetson, LiDAR, navigation, télémétrie	Organisation des topics robot : `/cmd_vel`, `/scan`, `/odom`, états et événements.
UART / pont série	Jetson – STM32	Transmission des commandes moteurs et réception de l'état firmware.
PWM/DIR	STM32 – drivers moteurs	Commande physique des moteurs de l'AGV.
USB / caméra / LiDAR	Capteurs – Jetson ou ROS 2	Acquisition du flux vidéo, des détections YOLO et des scans de l'environnement.

3.1.6 Communication Jetson–STM32

La liaison Jetson–STM32 est la liaison principale pour le mouvement. La Jetson envoie les commandes de vitesse, et la STM32 renvoie les données d'état.

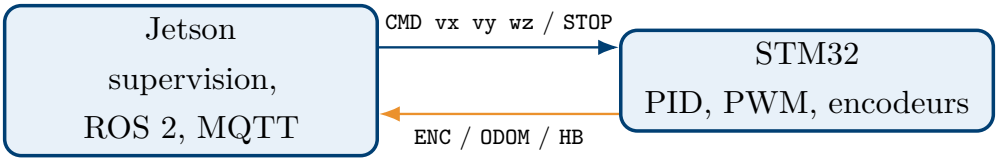


FIGURE 3.4 – Échanges série entre Jetson et STM32

3.1.7 Communication MQTT et Node-RED

MQTT est utilisé comme bus léger pour publier les commandes, les événements et la télémétrie. Node-RED complète MQTT en jouant le rôle de **broker logique** : il reçoit une action issue du dashboard, du voice agent ou d'un scénario de mission, puis il la transforme en message normalisé avant de la transmettre vers MQTT, rosbridge ou ROS 2. Cette organisation évite de connecter directement toutes les interfaces au robot et permet de modifier la logique de mission sans changer le firmware.

TABLE 3.3 – Topics MQTT proposés pour la supervision et les missions AGV

Topic	**Émetteur principal**	**Description**
`agv-bot`	Node-RED / robot / voice agent	Topic de mission utilisé pour échanger des messages simples comme `mission1`, `arrive` ou `piece_deposee`.
`agv/cmd_vel`	Dashboard / voice agent	Commande de vitesse normalisée.
`agv/mission`	Node-RED / voice agent	Lancement ou arrêt d'une mission automatique.
`agv/status`	Jetson / STM32	État général du robot : mode, disponibilité, erreur, arrêt d'urgence.
`agv/odom`	STM32 / Jetson	Position estimée, vitesse et informations de déplacement.
`agv/detection/person`	YOLO	Détection de personne ou d'objet visible par la caméra.
`agv/vision/description`	Jetson / voice agent	Résumé textuel de ce que le robot voit.
`agv/voice/intent`	Voice agent	Intention comprise après analyse de la commande vocale.
`agv/hololens/command`	HoloLens / Blazor	Commande envoyée depuis l'interface immersive.
`agv/emergency_stop`	Dashboard / voice agent	Arrêt d'urgence logiciel prioritaire.
`agv/nav/pose`	Jetson / Gazebo	Pose pour navigation et jumeau numérique.
`agv/nav/scan`	LiDAR / Gazebo	Scan simplifié envoyé aux interfaces web.
`agv/twin/state`	Jetson / Gazebo	État complet du jumeau numérique.

3.1.8 Exemple complet : commande « avance »

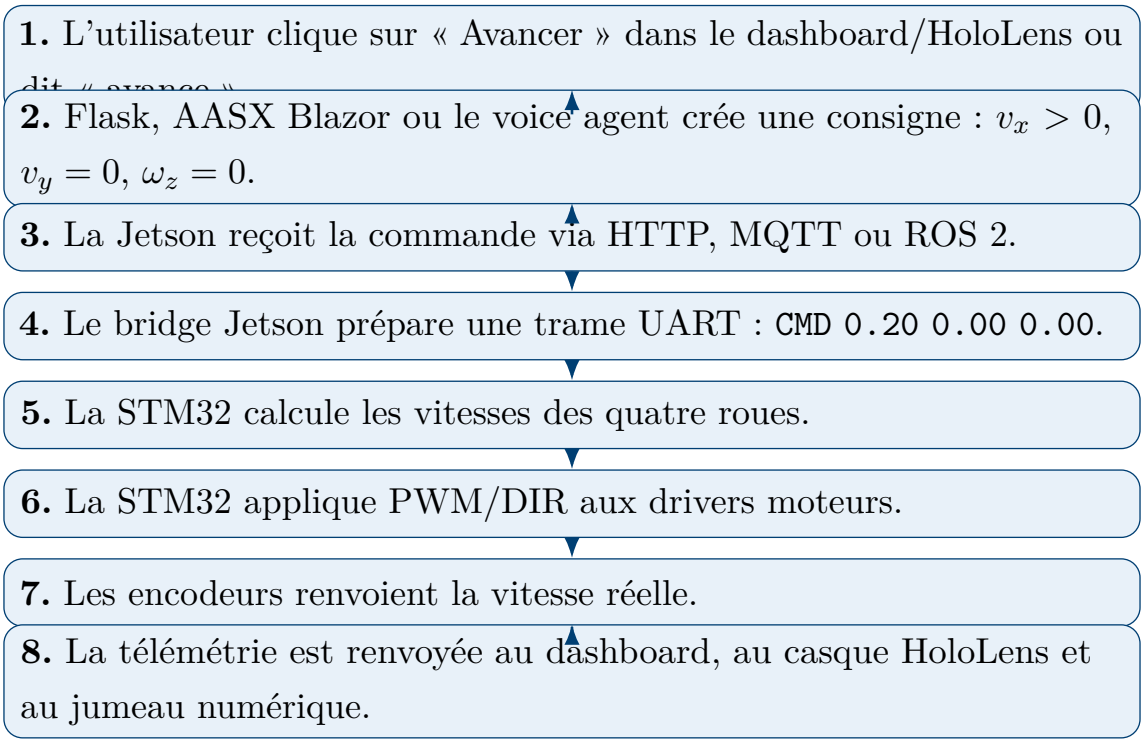


FIGURE 3.5 – Séquence de communication pour une commande d'avancement

3.1.9 Bilan

Bilan de communication cobotique

Le dashboard, le casque HoloLens, le jumeau numérique AASX Blazor et le voice agent représentent l'interface cobotique avec l'opérateur. La Jetson centralise la supervision, les services réseau et l'intelligence. La STM32 exécute la commande moteur temps réel. Les drivers alimentent les moteurs et les capteurs renvoient les informations nécessaires à la décision, au contrôle et à la sécurité.

3.2 Voice agent basé sur Gemini API

3.2.1 Objectif du voice agent

Le voice agent permet de commander le robot par la voix et de dialoguer avec l'utilisateur à propos de l'état du système. L'utilisateur peut donner une instruction naturelle, par exemple : « avance », « arrête-toi », « tourne à gauche », « lance la mission 1 » ou « qu'est-ce que tu vois devant toi ? ». Le système transforme ensuite cette phrase en intention

exploitable par le robot AGV.

Dans ce projet, le voice agent utilise une **clé API Gemini**. Cette clé permet au serveur vocal de communiquer avec Gemini API afin d'interpréter la commande de l'utilisateur, de produire une intention claire et de formuler une réponse lisible à partir des informations de télémétrie ou des détections YOLO.

3.2.2 Principe général du voice agent

Le voice agent est une couche intermédiaire entre l'utilisateur, la vision caméra et le robot. Il reçoit la voix, l'envoie vers Gemini API, récupère l'intention, consulte les informations YOLO si la demande concerne ce que le robot voit, puis transmet une commande technique vers la Jetson.

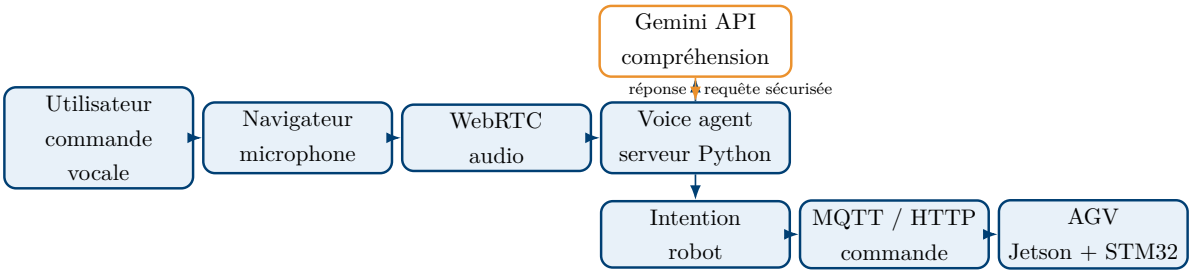


FIGURE 3.6 – Architecture du voice agent connecté à Gemini API et à la Jetson

3.2.3 Rôle de la clé API Gemini

La clé API permet d'autoriser les requêtes du voice agent vers Gemini API. Elle ne doit pas être écrite directement dans le code source public ni dans l'interface web. Elle doit être stockée dans un fichier local, par exemple `.env`, ou dans une variable d'environnement de la Jetson.

Listing 3.1 – Exemple de fichier .env sans exposer la vraie clé

```
GEMINI_API_KEY=mettre_votre_cle_api_ici
AGV_MQTT_HOST=localhost
AGV_MQTT_PORT=1883
AGV_CMD_TOPIC=agv/cmd_vel
AGV_MISSION_TOPIC=agv/mission
AGV_STOP_TOPIC=agv/emergency_stop
```

Sécurité de la clé API

La clé Gemini ne doit jamais être partagée dans le rapport, dans GitHub ou dans le code JavaScript côté navigateur. Elle doit rester uniquement côté serveur.

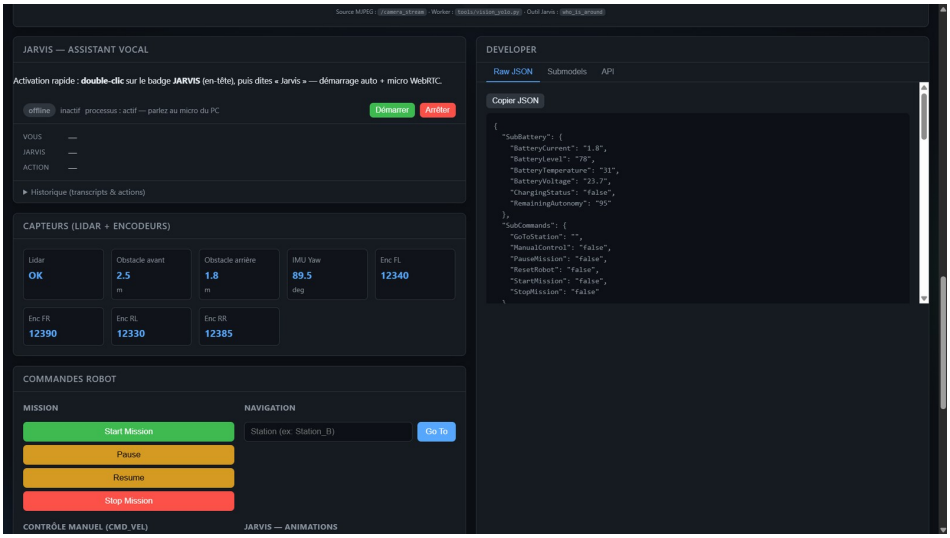


FIGURE 3.7 – Dashboard JARVIS : assistant vocal, capteurs et données JSON utilisées pour la supervision

3.2.4 Lien entre le voice agent et la détection caméra YOLO

La caméra connectée à la Jetson fournit un flux vidéo traité par YOLO. Les résultats de détection sont publiés sous forme de classes, scores de confiance et positions dans l'image. Le voice agent peut utiliser ces informations pour répondre à l'utilisateur ou adapter une mission.

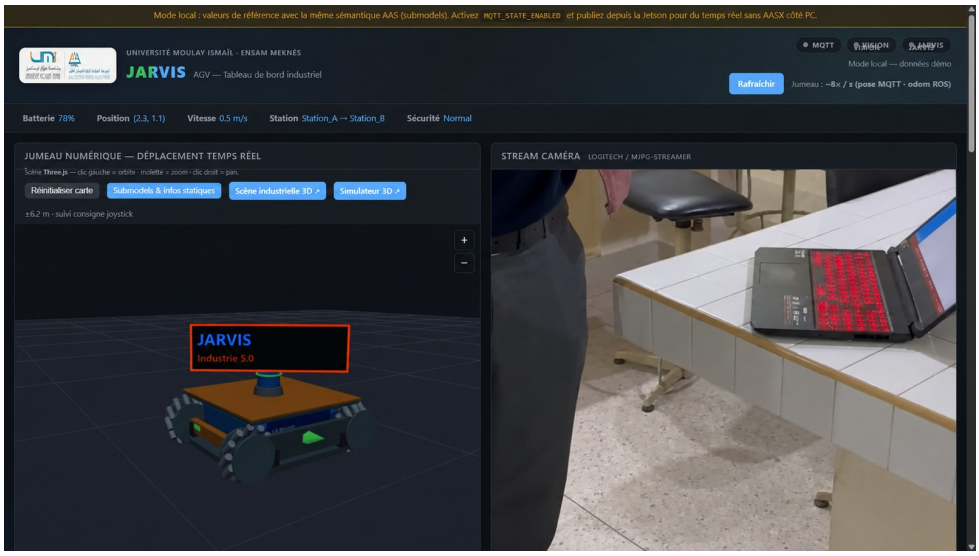


FIGURE 3.8 – Flux caméra intégré au dashboard et exploité par le voice agent pour décrire l'environnement

TABLE 3.4 – Exemples d'interaction entre la vision YOLO et le voice agent

Demande utilisateur	Information utilisée	Réponse ou action attendue
« Qu'est-ce que tu vois ? »	Classes YOLO détectées	Le système annonce les objets/personnes visibles
« Avance vers la zone libre »	Détection obstacle + LiDAR	Le robot vérifie la sécurité avant mouvement
« Lance la mission de transport »	Mode robot + zone de mission	Publication d'une mission MQTT validée
« Arrête-toi si tu vois une personne »	Détection personne	Arrêt ou ralentissement selon la logique de sécurité

Filtrage des commandes vocales

Une commande vocale qui dépend de la caméra ne doit pas être exécutée uniquement sur une phrase naturelle. Le système doit vérifier l'état du robot, la confiance de YOLO, les distances LiDAR et la priorité de l'arrêt d'urgence avant d'envoyer une consigne moteur.

3.2.5 Séquence de traitement d'une commande vocale

1. L'utilisateur dit : « avance doucement ».

2. Le casque HoloLens ou le navigateur transmet l'audio au voice agent par WebRTC.

3. Le voice agent interroge Gemini API à l'aide de la clé API.

4. Gemini retourne une intention : mouvement avant à faible vitesse.

5. Le voice agent convertit l'intention en commande : `CMD 0.20 0.00 0.00`

6. La commande est envoyée à la Jetson via MQTT, HTTP ou le pont AASX Blazor.

7. La Jetson transmet la commande à la STM32 par UART.

FIGURE 3.9 – Traitement complet d'une commande vocale depuis HoloLens ou navigateur

3.2.6 Conversion des phrases en commandes

TABLE 3.5 – Exemples d'intentions vocales, réponses et commandes associées

Phrase utilisateur	Intention détectée	Commande envoyée
« avance »	Déplacement avant	`CMD 0.20 0.00 0.00`
« recule »	Déplacement arrière	`CMD -0.20 0.00 0.00`
« va à gauche »	Translation gauche	`CMD 0.00 0.20 0.00`
« tourne à droite »	Rotation droite	`CMD 0.00 0.00 -0.40`
« arrête-toi »	Arrêt immédiat	`STOP`
« lance la mission 1 »	Mission automatique	`agv/mission = mission1`
« qu'est-ce que tu vois ? »	Interrogation vision	Lecture de `agv/vision/description`

3.2.7 Sécurité du voice agent

Avant d'envoyer une commande au robot, le voice agent doit vérifier que l'intention est autorisée. Une commande ambiguë doit être refusée ou reformulée. L'arrêt d'urgence doit rester prioritaire sur toutes les commandes vocales.

Bilan du voice agent

Gemini API sert à comprendre la demande vocale de l'utilisateur. Le voice agent transforme ensuite cette demande en commande robot normalisée, qui est transmise à la Jetson via MQTT ou HTTP, puis à la STM32 par UART.

3.3 Navigation autonome, simulation Gazebo et jumeau numérique Blazor

3.3.1 Objectif de la couche navigation et simulation

Au-delà du pilotage manuel et de la supervision temps réel, le projet intègre une couche de **navigation autonome** basée sur ROS 2, complétée par une **simulation Gazebo** permettant de valider localement les scans LiDAR, les corrections de trajectoire et les algorithmes de localisation avant déploiement sur le robot réel.

Un **jumeau numérique** développé en **AASX ASP.NET Blazor** est connecté à une

interface 3D pour visualiser en temps réel les mouvements du robot dans une scène d'usine, que ceux-ci proviennent de la simulation Gazebo ou de la plateforme physique embarquée sur Jetson.

Intérêt de l'approche

La combinaison ROS 2 + Gazebo + jumeau numérique Blazor permet de tester les scénarios de navigation en local, de corriger les paramètres (SLAM, contrôle, évitement) sans risque pour le matériel, puis de transférer la configuration validée vers le robot réel tout en supervisant les deux mondes depuis une interface unifiée.

3.3.2 Architecture ROS 2 pour la navigation autonome

Stack logicielle retenue

La navigation autonome s'appuie sur l'écosystème ROS 2 Humble exécuté sur la NVIDIA Jetson. Les composants principaux sont résumés dans le tableau 3.6.

TABLE 3.6 – Composants ROS 2 de la navigation autonome

Composant	Package / nœud	Rôle
Description robot	`robot_state_publisher`	Publie le modèle URDF/Xacro de l'AGV
Capteur LiDAR	`rplidar_ros` / driver D500	Publie `/scan`
Localisation	`slam_toolbox` ou `amcl`	Cartographie et pose dans la carte
Navigation	`nav2_bringup`	Planification globale et locale, contrôle
Contrôle vitesse	`twist_mux`	Priorise manuel / autonome / sécurité
Bridge embarqué	`agv_stm32_bridge`	Convertit `/cmd_vel` en trames UART
Visualisation	`rviz2`	Debug local des scans et trajectoires

Chaîne de navigation autonome

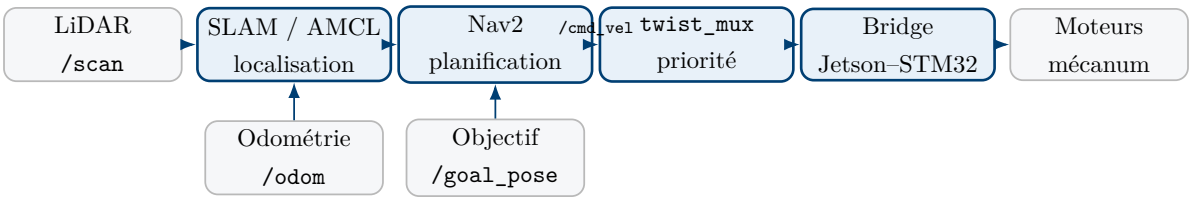


FIGURE 3.10 – Chaîne ROS 2 de la navigation autonome

Le nœud Nav2 calcule une trajectoire à partir de la carte, de la pose estimée et d'un objectif. La consigne de vitesse `/cmd_vel` est ensuite transmise au bridge embarqué, qui la convertit en commandes UART pour la STM32. Le `twist_mux` garantit que l'arrêt d'urgence ou le pilotage manuel depuis le dashboard reste prioritaire sur la navigation autonome.

Topics ROS 2 utilisés pour la navigation

TABLE 3.7 – Topics ROS 2 principaux de la navigation

Topic	Type	Description
`/scan`	`LaserScan`	Nuage de points LiDAR pour SLAM et évitement
`/odom`	`Odometry`	Pose et vitesse estimées (encodeurs + fusion)
`/map`	`OccupancyGrid`	Carte d'occupation générée par SLAM
`/goal_pose`	`PoseStamped`	Objectif de navigation
`/cmd_vel`	`Twist`	Consigne de vitesse (v_x, v_y, ω_z)
`/plan`	`Path`	Trajectoire planifiée par Nav2
`/tf`	`Transform`	Repères `map`, `odom`, `base_link`

3.3.3 Simulation Gazebo et validation locale

Environnement de simulation

Gazebo permet de reproduire le robot AGV, son LiDAR et l'environnement de test avant toute expérimentation sur le robot physique. Un modèle URDF/Xacro du châssis mécanum, des roues et du capteur LiDAR est chargé dans Gazebo via `ros2 launch agv_gazebo sim.launch.py`.

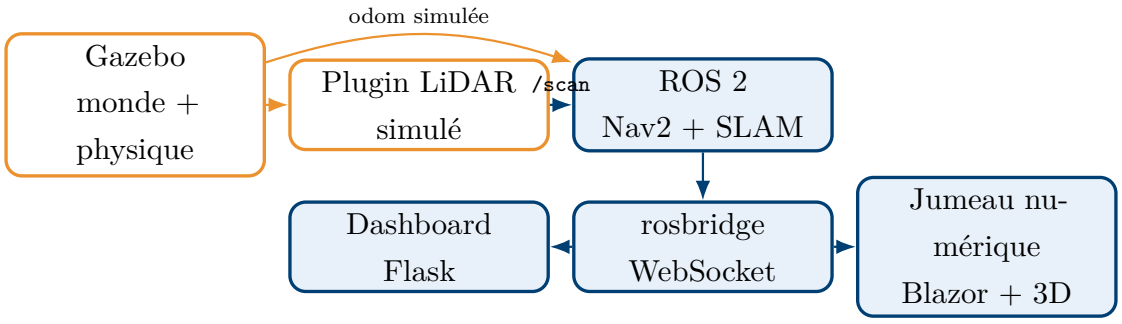


FIGURE 3.11 – Architecture de simulation Gazebo et diffusion vers les interfaces

Scans locaux et corrections de trajectoire

En simulation, le LiDAR virtuel publie des scans sur `/scan` à une fréquence configurable (typiquement 10 Hz). Ces données alimentent `slam_toolbox` pour construire la carte locale et corriger la pose du robot. Nav2 utilise ensuite la carte pour recalculer la trajectoire en cas d'obstacle détecté.

Les corrections locales testées en Gazebo incluent :

- **recalage SLAM** : ajustement de la pose lors d'une dérive odométrique simulée ;
- **recalcul de trajectoire** : le contrôleur local DWB ou RPP corrige la consigne pour contourner un obstacle ;
- **arrêt de sécurité** : si la distance minimale devant le robot devient inférieure au seuil configuré, Nav2 publie une consigne nulle ;
- **validation des paramètres** : vitesses max, accélérations et rayon de rotation sont ajustés avant transfert vers le robot réel.

Listing 3.2 – Lancement de la simulation Gazebo avec navigation

```
# Terminal 1 : simulation Gazebo + robot
ros2 launch agv_gazebo sim.launch.py

# Terminal 2 : navigation Nav2
ros2 launch agv_navigation nav2_bringup.launch.py use_sim_time:=true

# Terminal 3 : SLAM en mode mapping
ros2 launch slam_toolbox online_async_launch.py use_sim_time:=true
```

Transfert simulation vers robot réel

Une fois les paramètres validés en Gazebo (carte, vitesses, seuils d'évitement), la même configuration est déployée sur la Jetson avec `use_sim_time:=false`. Le fichier de paramètres Nav2 (`nav2_params.yaml`) et la carte SLAM exportée (`map.yaml`) sont réutilisés sans modification majeure, ce qui réduit les risques lors des essais réels.

3.3.4 Envoi des données vers l'interface de supervision

Pipeline de données

Les données produites par ROS 2 (scans, odométrie, carte, trajectoire, état Nav2) sont relayées vers le dashboard Flask et le jumeau numérique Blazor via une chaîne de ponts réseau.

1. ROS 2 publie `/scan`, `/odom`, `/map`, `/plan`, `/robot_status`.

2. `rosbridge_server` expose les topics en WebSocket, port 9090.

3. Un nœud `agv_mqtt_bridge` republie les données vers MQTT.

4. Le dashboard Flask consomme MQTT et WebSocket pour afficher LiDAR et télémétrie.

5. Le serveur Blazor reçoit les mêmes flux via SignalR et MQTT.

6. L'interface 3D met à jour la pose et le scan du robot en temps réel.

FIGURE 3.12 – Pipeline d'envoi des données ROS 2 vers les interfaces utilisateur

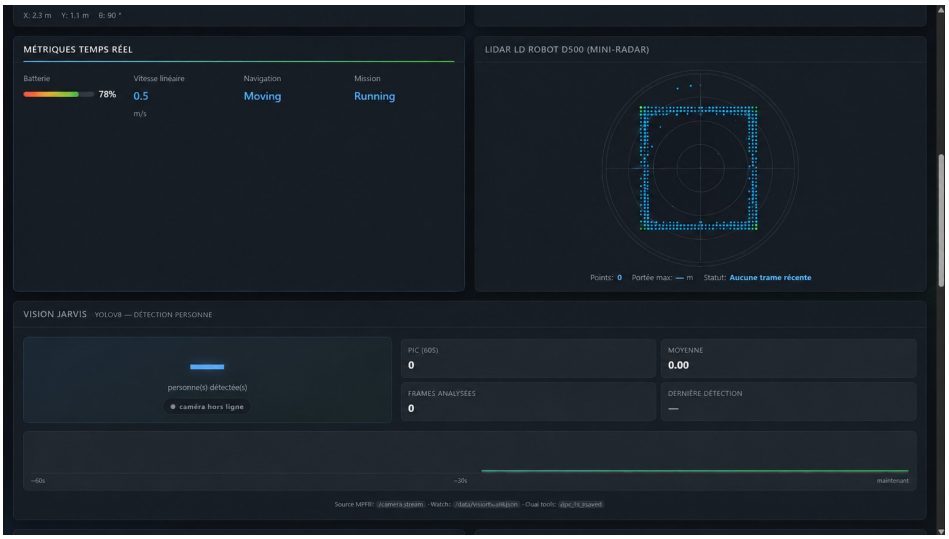


FIGURE 3.13 – Visualisation du scan LiDAR dans le dashboard : points organisés sous forme de carré représentant les obstacles détectés

Topics MQTT dédiés à la navigation

Table 3.8 – Topics MQTT pour la navigation et le jumeau numérique

Topic	Format	Description
`agv/nav/pose`	JSON `{x,y,yaw}`	Pose courante du robot
`agv/nav/scan`	JSON array	Scan LiDAR simplifié pour le web
`agv/nav/map`	PNG/base64 ou tuile	Carte d'occupation
`agv/nav/path`	JSON waypoints	Trajectoire planifiée Nav2
`agv/nav/status`	string	État Nav2 : idle, navigating, recovered
`agv/nav/goal`	JSON `{x,y,yaw}`	Objectif envoyé depuis l'interface
`agv/twin/state`	JSON complet	État jumeau numérique (pose + vitesses + mode)

Listing 3.3 – Exemple de publication MQTT depuis un nœud ROS 2

```python
import json, paho.mqtt.client as mqtt

def publish_pose(pose):
    payload = {
        "x": pose.position.x,
        "y": pose.position.y,
        "yaw": yaw_from_quaternion(pose.orientation),
        "source": "ros2_nav"
    }
    client.publish("agv/nav/pose", json.dumps(payload), qos=1)

def publish_scan(scan_msg):
    payload = {
        "angle_min": scan_msg.angle_min,
        "angle_increment": scan_msg.angle_increment,
        "ranges": list(scan_msg.ranges[:360])
    }
    client.publish("agv/nav/scan", json.dumps(payload), qos=0)
```

3.3.5 Jumeau numérique AASX ASP.NET Blazor

Principe du jumeau numérique

Le jumeau numérique est une application **AASX ASP.NET Blazor Server** qui reproduit virtuellement l'état du robot AGV : position, orientation, vitesses, scan LiDAR, trajectoire planifiée et mode de fonctionnement (manuel, autonome, simulation Gazebo).

Il constitue une couche de supervision avancée complémentaire au dashboard Flask, avec une visualisation 3D interactive des mouvements du robot. Dans l'usage HoloLens, cette interface devient un cockpit immersif : l'opérateur voit le modèle AGV, la trajectoire, les obstacles détectés et l'état de mission dans une scène représentant l'usine.

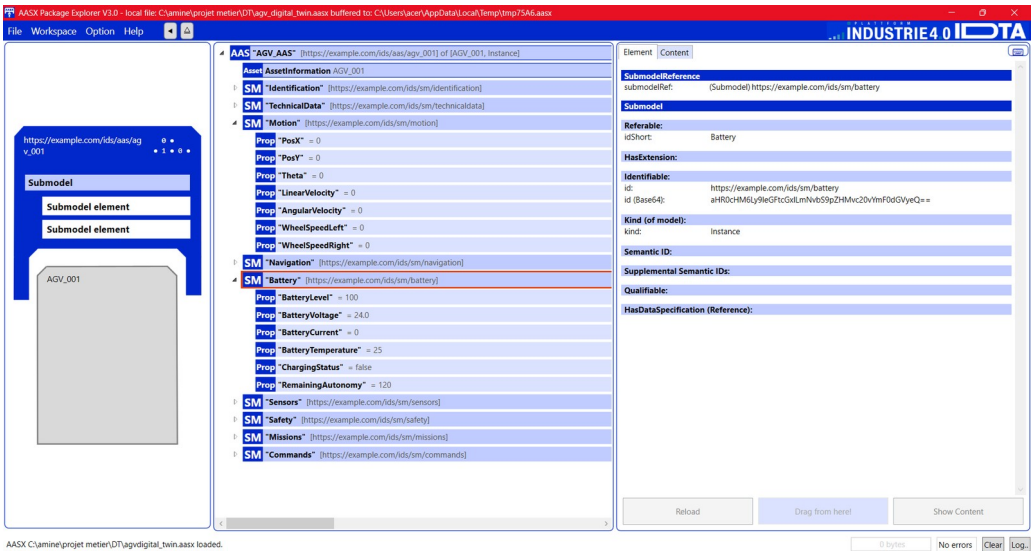


FIGURE 3.14 – Modèle AASX du robot AGV : organisation des submodels Identification, Motion, Navigation, Battery, Sensors, Safety, Missions et Commands

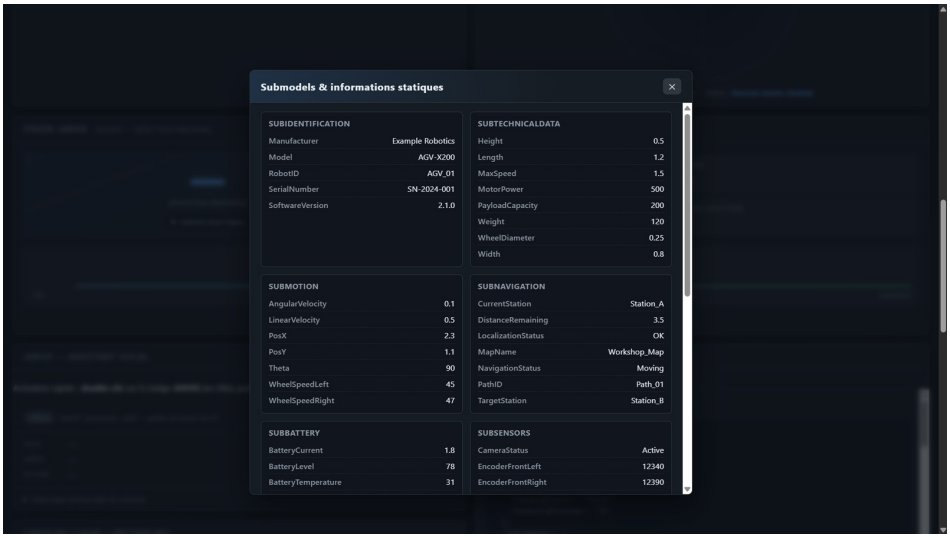


FIGURE 3.15 – Affichage des submodels et des informations statiques dans le dashboard AASX Blazor



FIGURE 3.16 – Scène 3D de l'usine dans le jumeau numérique AASX Blazor

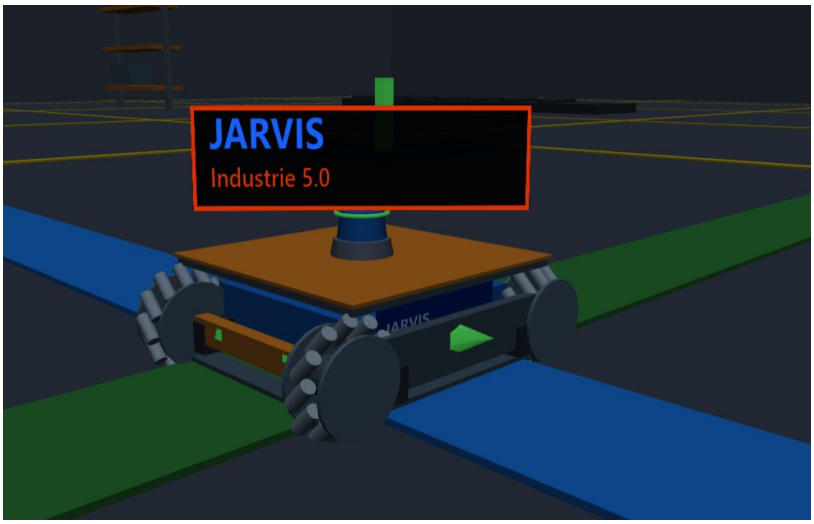


FIGURE 3.17 – Vue rapprochée du modèle AGV JARVIS dans la scène 3D du jumeau numérique

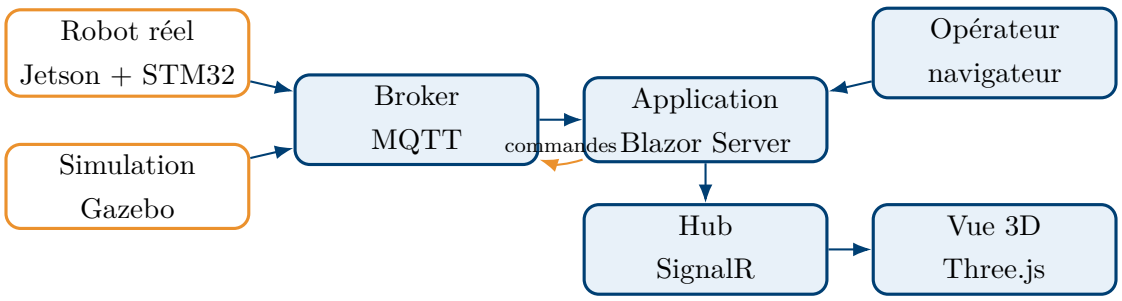


FIGURE 3.18 – Architecture du jumeau numérique Blazor connecté au robot et à Gazebo

Structure de l'application Blazor

L'application est organisée en composants réutilisables :

TABLE 3.9 – Composants principaux du jumeau numérique Blazor

Composant	Technologie	Rôle
`TwinDashboard`	Blazor Server	Page principale de supervision
`RobotScene3D`	Three.js (JS interop)	Scène 3D : robot, sol, obstacles
`LidarOverlay`	Canvas 2D	Projection du scan LiDAR
`MapPanel`	Blazor + image	Affichage carte SLAM
`TelemetryPanel`	Blazor	Vitesses, batterie, mode Nav2
`MqttTwinService.cs`	MQTTnet	Abonnement aux topics `agv/twin/*`
`TwinHub.cs`	SignalR	Push temps réel vers le navigateur

Communication temps réel avec SignalR

SignalR permet de pousser les mises à jour de pose et de scan vers le navigateur sans polling. Lorsqu'un message MQTT `agv/twin/state` est reçu, le service `MqttTwinService` notifie le hub `TwinHub`, qui diffuse l'état à tous les clients connectés.

Listing 3.4 – Hub SignalR pour la mise à jour du jumeau numérique (extrait C#)

```
public class TwinHub : Hub
{
    public async Task SendRobotState(RobotTwinState state)
    {
        await Clients.All.SendAsync("RobotStateUpdated", state);
    }
}

public class MqttTwinService : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken token)
    {
        await mqttClient.SubscribeAsync("agv/twin/state");
        mqttClient.ApplicationMessageReceivedAsync += async e =>
        {
            var state = JsonSerializer.Deserialize<RobotTwinState>(
```

```
                e.ApplicationMessage.ConvertPayloadToString());
            await twinHub.Clients.All.SendAsync("RobotStateUpdated", state);
        };
    }
}
```

3.3.6 Interface 3D de visualisation des mouvements

Scène Three.js intégrée à Blazor

L'interface 3D est implémentée avec **Three.js**, appelé depuis Blazor via `IJSRuntime`. La scène comprend :

- un modèle 3D simplifié du châssis AGV et des quatre roues mécanum ;
- un plan au sol représentant l'environnement ou la carte SLAM ;
- des points LiDAR colorés selon la distance ;
- une polyligne représentant la trajectoire planifiée par Nav2 ;
- un repère `map` / `odom` / `base_link` pour le debug.

La pose du robot est mise à jour à chaque réception SignalR : translation (x, y), rotation θ autour de l'axe vertical, et interpolation linéaire entre deux états pour un rendu fluide à 30 Hz. Lorsque l'interface est affichée dans HoloLens, les mêmes informations sont projetées dans la vue opérateur afin de suivre les mouvements du robot dans le contexte de l'usine.

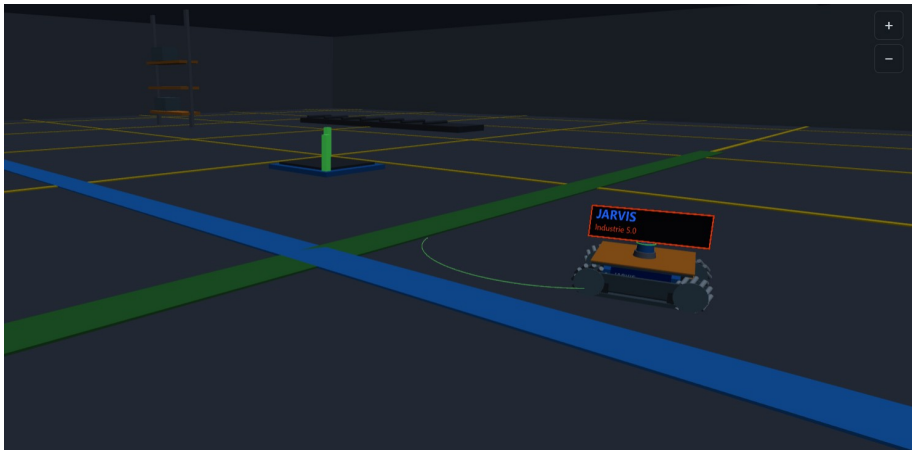


FIGURE 3.19 – la scène 3D Three.j

Synchronisation robot réel / simulation

Le jumeau numérique distingue la source des données grâce au champ `source` du message JSON (`real`, `gazebo`, `replay`). L'opérateur peut basculer entre :

1. **mode réel** : données MQTT provenant de la Jetson embarquée ;
2. **mode simulation** : données Gazebo relayées par le même broker MQTT ;
3. **mode comparaison** : superposition des trajectoires réelle et simulée pour valider la cohérence des paramètres Nav2.

Cohérence des repères

Les repères `map`, `odom` et `base_link` doivent être identiques entre Gazebo, ROS 2, le dashboard Flask et le jumeau numérique Blazor. Un décalage de repère provoquerait une visualisation 3D incorrecte des mouvements.

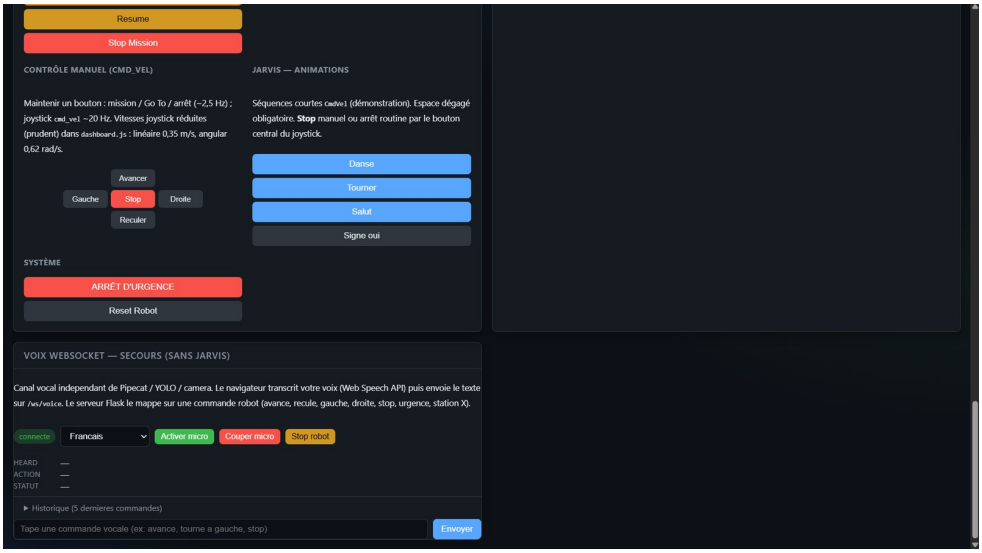


FIGURE 3.20 – Commande manuelle du robot, missions et canal vocal WebSocket depuis le dashboard affiché dans HoloLens

3.3.7 Commande de navigation depuis le jumeau numérique

L'opérateur peut envoyer un objectif de navigation depuis l'interface Blazor en cliquant sur la carte 2D ou en saisissant des coordonnées. La commande transite par MQTT (`agv/nav/goal`), est reçue par un nœud ROS 2 `goal_relay` qui publie sur `/goal_pose`, puis Nav2 exécute la mission.

Listing 3.5 – Envoi d'un objectif de navigation depuis Blazor (extrait)

```
1 private async Task SendNavigationGoal(double x, double y, double yaw)
```

```
{
    var goal = new { x, y, yaw, frame = "map" };
    await mqttService.PublishAsync("agv/nav/goal", JsonSerializer.Serialize(
    goal));
    statusMessage = $"Objectif envoye : ({x:F2}, {y:F2})";
}
```

3.3.8 Bilan

Apport de la navigation et du jumeau numérique

La couche ROS 2 + Nav2 apporte la navigation autonome avec localisation SLAM et évitement d'obstacles. Gazebo permet de valider localement les scans, les corrections de trajectoire et les paramètres avant essai réel. Le jumeau numérique AASX ASP.NET Blazor, couplé à une interface 3D Three.js et alimenté par MQTT/SignalR, offre une visualisation temps réel des mouvements du robot, que ceux-ci proviennent de la simulation ou de la plateforme physique.

4. Positionnement du projet selon le modèle RAMI 4.0

4.1 Pourquoi utiliser RAMI 4.0 ?

Le modèle **RAMI 4.0** (*Reference Architectural Model Industrie 4.0*) est une architecture de référence utilisée pour structurer les systèmes industriels intelligents. Il permet de positionner un projet selon trois axes complémentaires : les **couches fonctionnelles**, les **niveaux hiérarchiques** de l'usine et le **cycle de vie / chaîne de valeur**. Cette représentation est particulièrement utile pour montrer qu'un projet ne se limite pas à un robot physique, mais couvre également les échanges de données, les services logiciels, les fonctions intelligentes et les interfaces opérateur [1, 2].

Dans le cas de notre AGV JARVIS, le recours à RAMI 4.0 est pertinent car le projet associe :

— un **actif physique** : le robot AGV, les moteurs, la STM32, le LiDAR, la caméra et les encodeurs ;

— une **couche d'intégration** : Jetson Orin Nano, acquisition capteurs, commande temps réel et actionneurs ;

— une **couche de communication** : UART, MQTT, ROS 2, WebSocket, HTTP/-REST et MJPG-streamer ;

— une **couche informationnelle** : AAS/AASX, JSON temps réel, télémétrie, états et sous-modèles ;

— une **couche fonctionnelle** : dashboard, jumeau numérique 3D, YOLO, Node-RED, commandes manuelles et vocales ;

— une **couche business / supervision** : gestion des missions, maintenance, traçabilité, décision opérateur et vision Industrie 5.0.

4.2 Architecture du projet selon RAMI 4.0

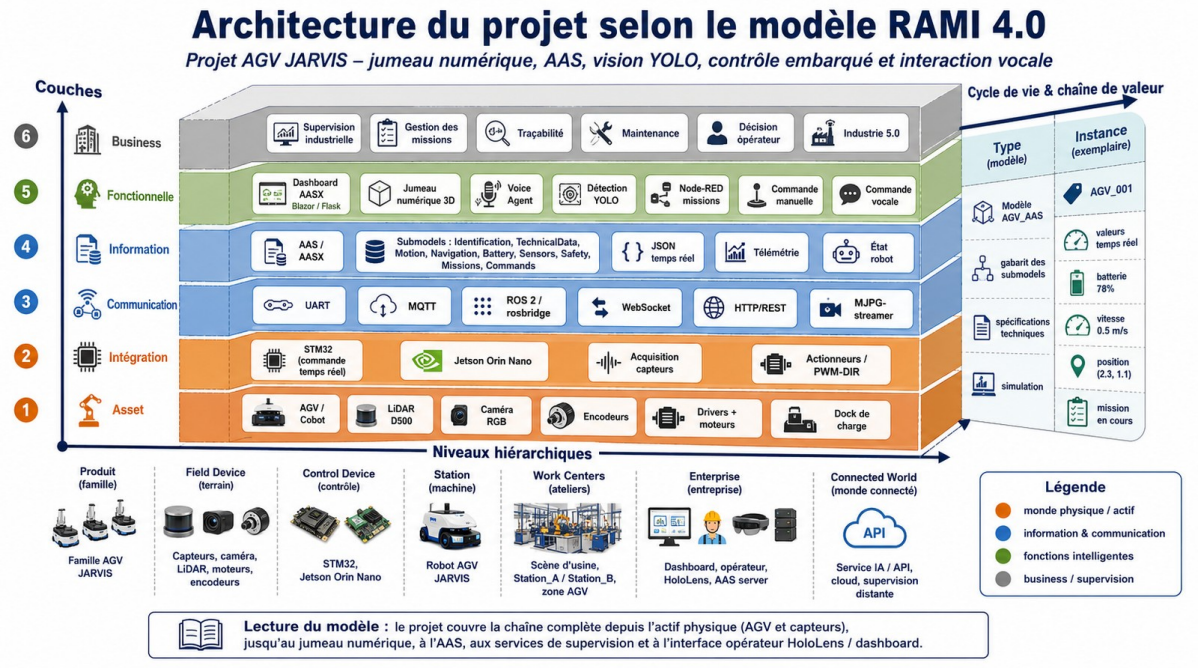


FIGURE 4.1 – Architecture du projet AGV JARVIS positionnée selon les axes du modèle RAMI 4.0

La figure 4.1 montre que le projet couvre pratiquement toute la chaîne fonctionnelle, depuis le robot réel jusqu'aux services de supervision avancés. Cette représentation met en évidence le fait que le jumeau numérique, l'AAS et l'interface opérateur ne sont pas des éléments isolés ; ils constituent une continuité logique avec la partie embarquée et la partie réseau.

4.3 Lecture par couches

4.3.1 Couche 1 – Asset

La couche **Asset** regroupe tous les éléments physiques du système. Dans notre projet, il s'agit de l'AGV, des roues motrices, des encodeurs, du LiDAR D500, de la caméra RGB, des drivers moteurs et du dock de charge. C'est la couche la plus concrète : elle correspond aux objets réellement installés dans l'environnement expérimental.

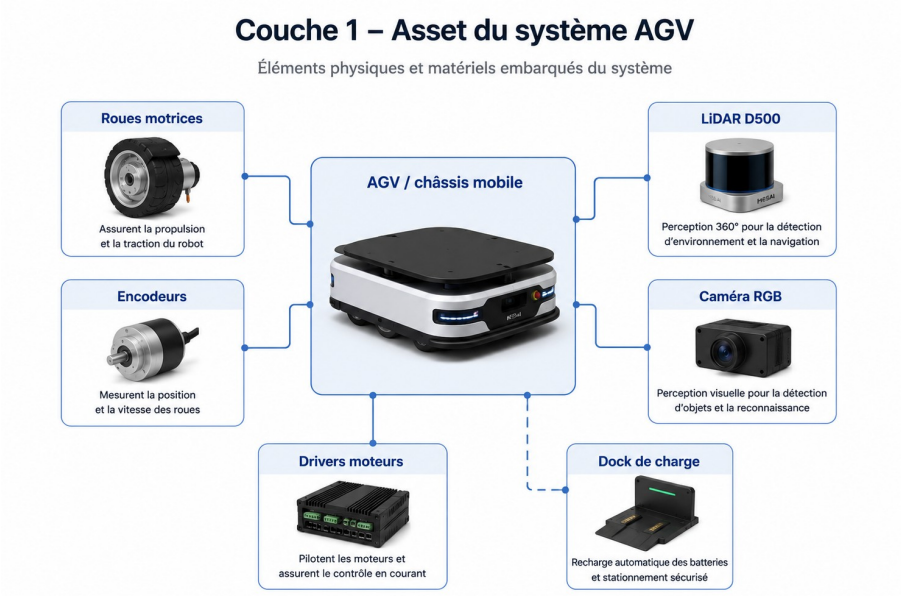
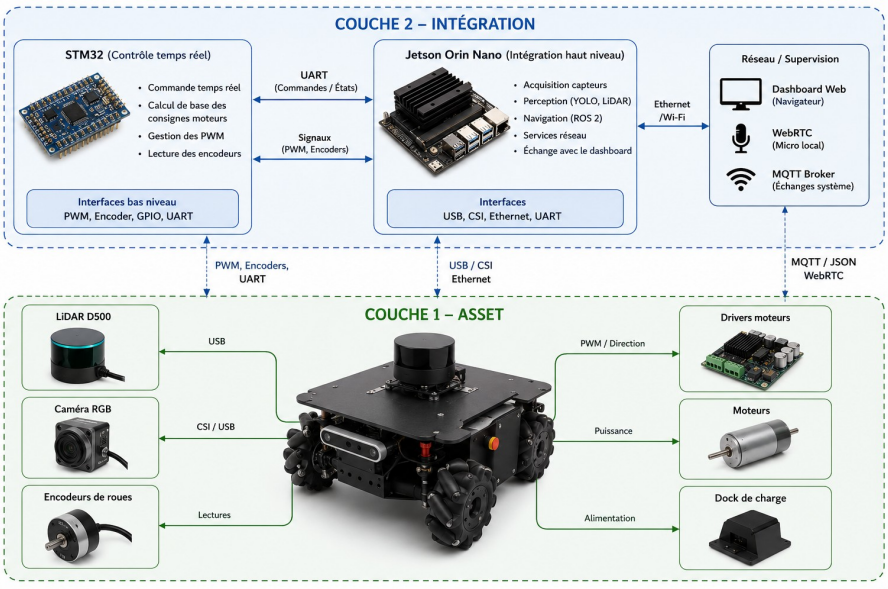


Figure — Représentation de la couche Asset regroupant les composants physiques du système AGV

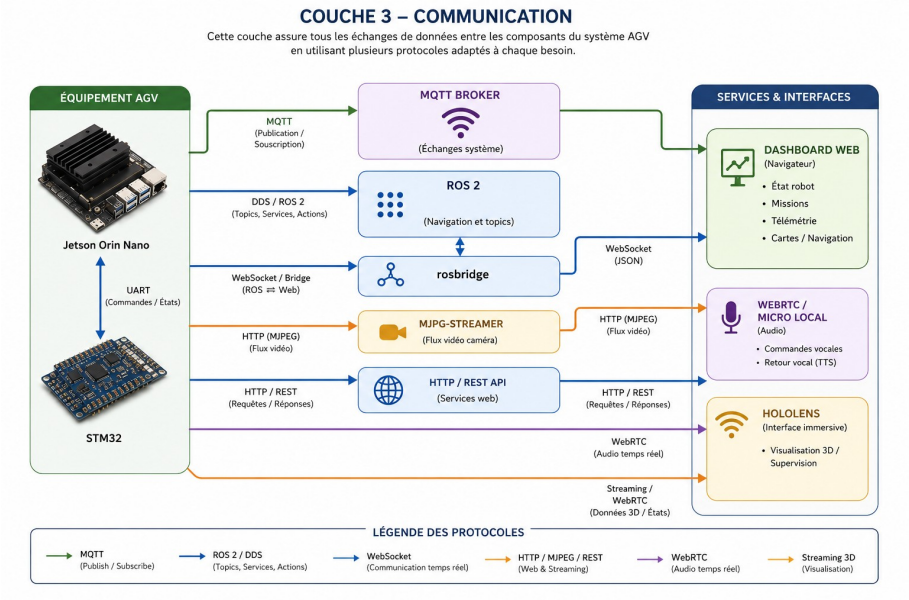
4.3.2 Couche 2 – Intégration

La couche **Intégration** assure le lien entre le monde physique et le monde numérique. La **STM32** prend en charge la commande temps réel, le calcul de base des consignes moteurs, la gestion des PWM et la lecture des encodeurs. La **Jetson Orin Nano**, quant à elle, assure l'intégration logicielle de plus haut niveau : acquisition capteurs, perception, navigation, services réseau et échange avec le dashboard. Cette séparation rend l'architecture plus robuste : la STM32 garantit la réactivité bas niveau, tandis que la Jetson se concentre sur l'intelligence et la supervision.



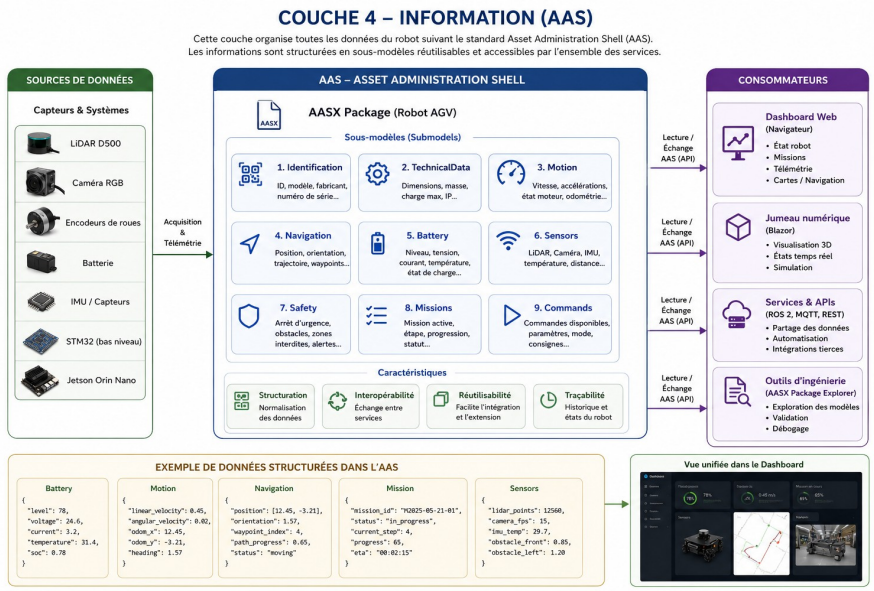
4.3.3 Couche 3 – Communication

La couche **Communication** est très importante dans ce projet, car l'AGV échange en permanence avec plusieurs services. La liaison **UART** permet le dialogue Jetson–STM32. Le protocole **MQTT** assure la publication et la souscription de messages légers pour l'état du robot, les missions et la télémétrie. **ROS 2** et **rosbridge** servent à la navigation, au partage des topics et à la connexion avec les interfaces web. **HTTP/REST** et **WebSocket** permettent l'accès aux tableaux de bord et aux mises à jour temps réel, tandis que **MJPG-streamer** transporte le flux caméra dans l'interface de supervision [8, 4].



4.3.4 Couche 4 – Information

La couche **Information** regroupe la structuration des données. Le projet utilise le concept d'**AAS** (*Asset Administration Shell*) pour organiser les informations du robot sous forme de sous-modèles : *Identification*, *TechnicalData*, *Motion*, *Navigation*, *Battery*, *Sensors*, *Safety*, *Missions* et *Commands*. Cette structuration facilite la lecture des données, leur partage entre services et leur réutilisation par le dashboard et le jumeau numérique [3].



Concrètement, cela signifie que les mesures du robot ne sont pas simplement des variables isolées. Elles sont organisées dans un modèle cohérent. Par exemple, le niveau de batterie, la position, la vitesse, l'état mission et les capteurs peuvent être exposés à la fois dans l'AASX Package Explorer, dans le dashboard et dans le jumeau numérique Blazor.

4.3.5 Couche 5 – Fonctionnelle

La couche **Fonctionnelle** correspond aux services qui donnent du sens aux données. Dans notre cas, on y trouve :

— le **dashboard AASX Blazor / Flask** pour la supervision ;

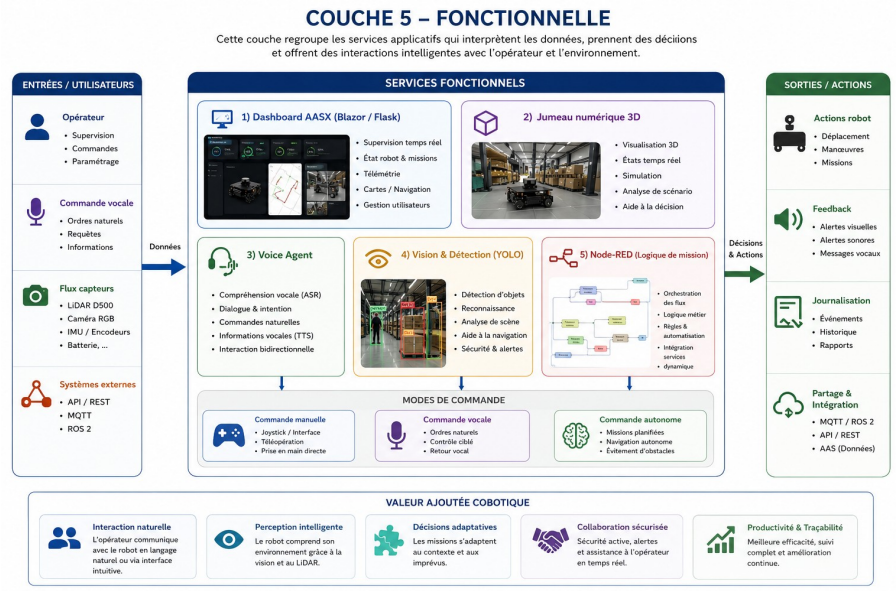
— le **jumeau numérique 3D** pour la visualisation spatiale ;

— le **voice agent** pour la commande naturelle ;

— la **détection YOLO** pour l'interprétation de la scène ;

— **Node-RED** pour la logique de mission ;

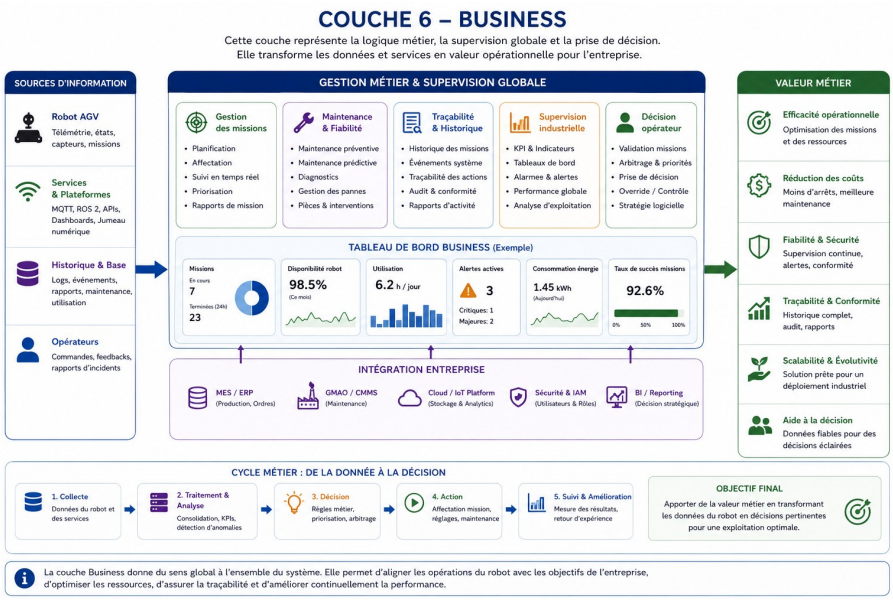
— les fonctions de **commande manuelle** et de **commande vocale**.



Cette couche montre bien l'orientation **cobotique** du projet : l'opérateur ne se contente pas d'envoyer un ordre, il dispose d'outils d'interaction, de compréhension et de visualisation du comportement robotique.

4.3.6 Couche 6 – Business

La couche **Business** est la couche la plus abstraite. Elle concerne les besoins métier, la supervision globale et la logique de décision. Dans le projet JARVIS, cette couche est représentée par la gestion des missions, la maintenance, la traçabilité, la supervision industrielle et la décision opérateur. Même si le robot reste une plateforme académique, cette couche montre comment la solution peut évoluer vers un usage industriel plus complet : suivi d'exploitation, maintenance prédictive, intégration MES/ERP ou indicateurs de performance.



4.4 Lecture selon les niveaux hiérarchiques

Le deuxième axe de RAMI 4.0 concerne les **niveaux hiérarchiques**. Dans notre projet, ils peuvent être interprétés comme suit :

— **Produit** : la famille AGV JARVIS ;

— **Field Device** : les capteurs, la caméra, le LiDAR, les encodeurs et les actionneurs ;

— **Control Device** : la STM32 et la Jetson Orin Nano ;

— **Station** : le robot AGV lui-même ;

— **Work Centers** : la scène d'usine, les stations de mission et la zone de circulation ;

— **Enterprise** : le dashboard, l'opérateur, le casque HoloLens et le serveur AAS ;

— **Connected World** : les services IA, les API, le cloud et la supervision distante.

Cette lecture hiérarchique montre que le projet ne se limite pas à une simple carte de contrôle. Il relie le niveau terrain au niveau connecté, ce qui est caractéristique des architectures Industrie 4.0.

4.5 Lecture selon le cycle de vie et la chaîne de valeur

Le troisième axe de RAMI 4.0 sépare la partie **Type** et la partie **Instance**. Cette distinction est très utile dans notre projet :

— la partie **Type** correspond au modèle général du robot AGV, à son AAS, à ses sous-modèles et à ses spécifications techniques ;

— la partie **Instance** correspond au robot `AGV_001` réellement utilisé pendant les essais, avec ses valeurs courantes : batterie à 78%, vitesse à 0,5 m/s, position (2.3, 1.1) et mission en cours.

Ce découpage type/instance est directement compatible avec l'idée d'un jumeau numérique : le **modèle** décrit la structure permanente du système, tandis que l'**instance** transporte les valeurs temps réel du robot observé [3].

4.6 Table de correspondance RAMI 4.0 / éléments du projet

TABLE 4.1 – Correspondance entre RAMI 4.0 et les composants du projet AGV JARVIS

Axe RAMI	**Élément du projet**	**Interprétation**
Asset	AGV, LiDAR, caméra, encodeurs, moteurs	Monde physique et capteurs réels
Intégration	STM32, Jetson, acquisition capteurs, PWM/DIR	Interface entre physique et numérique
Communication	UART, MQTT, ROS 2, WebSocket, HTTP/REST	Transport et synchronisation des données
Information	AAS/AASX, JSON, télémétrie, états robot	Structuration des données et interopérabilité
Fonctionnelle	Dashboard, jumeau numérique, YOLO, voice agent, Node-RED	Services intelligents et interaction homme–robot
Business	Supervision, maintenance, gestion missions, décision opérateur	Vue métier et exploitation industrielle
Hiérarchie	Field device à Connected World	Continuité du terrain vers le cloud
Cycle de vie	Type / Instance	Modèle AGV et robot réel AGV_001

4.7 Discussion

Le positionnement selon RAMI 4.0 montre que le projet possède déjà plusieurs caractéristiques d'une plateforme **Industrie 4.0** complète :

- une séparation claire entre le niveau physique, le niveau de communication et le niveau de service ;
- une représentation normalisée des données via l'AAS ;
- une ouverture vers des fonctions avancées comme la vision YOLO, la commande vocale et le jumeau numérique ;
- une capacité de montée en échelle, depuis un robot académique unique vers une flotte d'AGV supervisés.

Cependant, certaines extensions restent possibles pour se rapprocher encore plus d'un déploiement industriel : ajout d'une cybersécurité plus poussée, intégration à un MES/ERP, historisation industrielle des données, traçabilité plus fine des missions et gestion multi-robots.

Bilan du positionnement RAMI 4.0

L'analyse selon RAMI 4.0 confirme que le projet JARVIS n'est pas seulement un prototype mobile. Il s'agit d'une plateforme intégrée combinant actif physique, électronique de commande, communication temps réel, modèle de données AAS, services intelligents, jumeau numérique et interface opérateur immersive. Cette lecture renforce la cohérence architecturale du projet et met en évidence sa compatibilité avec les principes de l'Industrie 4.0.

5. Tests et validation

5.1 Objectif des essais

Les essais permettent de vérifier que chaque module fonctionne séparément, puis que le système complet se comporte correctement une fois intégré.

La validation suit une logique progressive :

1. validation électrique ;
2. validation firmware ;
3. validation communication ;
4. validation dashboard ;
5. validation HoloLens et interface AASX Blazor ;
6. validation capteurs ;
7. validation navigation autonome ROS 2 et simulation Gazebo ;
8. validation jumeau numérique Blazor et interface 3D ;
9. validation mouvement robot ;
10. validation globale avec mission.

5.2 Plan de test

TABLE 5.1 – Plan de test fonctionnel du système AGV

Test	**Méthode**	**Critère de validation**
Alimentation	Mesurer les tensions à vide et en charge	Tensions stables, pas de chute excessive
STM32	Envoyer des commandes simples par UART	Réponse correcte et absence de blocage

Test	**Méthode**	**Critère de validation**
PWM moteurs	Tester chaque moteur séparément	Sens et vitesse cohérents
Encodeurs	Faire tourner les roues et lire les impulsions	Comptage régulier et signe correct
PID	Appliquer une consigne de vitesse	Vitesse mesurée proche de la référence
Dashboard	Commander le robot depuis l'interface	Commandes reçues par la Jetson
HoloLens	Afficher le dashboard et le jumeau numérique	Interface lisible, commandes accessibles et retour état visible
AASX Blazor	Visualiser la scène 3D usine	Le modèle AGV suit les mouvements réels ou simulés
MQTT	Publier et souscrire aux topics	Messages reçus sans perte visible
ROS 2	Publier sur `/cmd_vel`	Mouvement ou transmission vers STM32
LiDAR	Lire le scan dans ROS 2	Données affichées et cohérentes
Caméra	Afficher le flux dans Flask	Image fluide et lisible
YOLO	Tester la détection de personnes/objets	Détection affichée dans le dashboard et transmise au voice agent
Arrêt urgence	Activer STOP matériel ou logiciel	Arrêt immédiat des moteurs
Navigation Nav2	Envoyer un objectif `/goal_pose`	Trajectoire calculée et robot en mouvement
SLAM Gazebo	Lancer `slam_toolbox` en simulation	Carte construite, pose cohérente
Scan simulé	Lire `/scan` depuis Gazebo	Données reçues dans RViz et dashboard
Jumeau numérique	Connecter AASX Blazor au broker MQTT	Pose 3D mise à jour en temps réel

Test	Méthode	Critère de validation
Interface 3D	Observer mouvement robot réel ou simulé	Modèle AGV suit trajectoire Nav2 et scène usine cohérente
Voice agent + YOLO	Demander ce que le robot voit	Réponse cohérente avec les classes détectées

5.3 Tests sous RViz avant la mise en mouvement du robot réel

Avant d'autoriser le robot à se déplacer, une étape de validation est réalisée dans **RViz**. Cette étape est essentielle car elle permet de vérifier que la chaîne ROS 2, la perception et la localisation sont cohérentes avant d'appliquer une commande moteur réelle [5, 6].

L'idée n'est pas seulement d'ouvrir RViz pour visualiser le robot, mais d'utiliser cet outil comme un **poste d'inspection de sécurité et de cohérence**. Si la pose, les repères, le scan LiDAR ou l'odométrie sont incohérents dans RViz, le robot ne doit pas être lancé en déplacement autonome ou manuel rapide.

5.3.1 Objectifs du test RViz

Les vérifications menées dans RViz avant la mise en marche sont les suivantes :

— vérifier la présence des repères `map`, `odom`, `base_link` et des repères capteurs ;

— vérifier que le modèle du robot est correctement affiché ;

— contrôler la cohérence du scan LiDAR dans l'environnement proche ;

— observer la publication de l'odométrie et de la trajectoire ;

— confirmer la bonne réception de `/cmd_vel` et de `/scan` ;

— s'assurer que la direction du déplacement demandé correspond à la direction observée dans RViz.

5.3.2 Procédure recommandée

Une procédure simple et sûre peut être adoptée :

1. démarrer la Jetson, la STM32 et les nœuds ROS 2 de base ;
2. lancer la publication des transformations `/tf` et le modèle URDF du robot ;
3. démarrer le driver LiDAR et vérifier la publication de `/scan` ;
4. ouvrir RViz avec les affichages *RobotModel*, *TF*, *LaserScan*, *Odometry*, *Path* et, si disponible, *Map* ;
5. faire un test statique puis un test à très faible vitesse ;
6. n'autoriser le déplacement réel qu'après validation visuelle et logique des informations.

5.3.3 Checklist de validation sous RViz

TABLE 5.2 – Checklist de validation RViz avant déplacement du robot

Élément contrôlé	**Observation dans RViz**	**Critère d'acceptation**
Arbre TF	Repères visibles et reliés	Chaîne `map -> odom -> base_link` cohérente
RobotModel	Modèle géométrique affiché	Orientation du robot conforme au réel
LaserScan	Points ou nuages autour du robot	Obstacles visibles, orientation du scan correcte
Odometry	Flèche / trajectoire évolutive	Variation cohérente avec le petit déplacement imposé
`/cmd_vel`	Commande envoyée au robot	Direction affichée compatible avec la consigne
Goal pose / Path	Objectif et trajectoire	Chemin réaliste, sans saut ni inversion brutale
Coût local / global	Zones d'obstacles et espace libre	Carte exploitable pour Nav2
Sécurité	Arrêt de test disponible	Possibilité d'arrêter immédiatement le robot

5.3.4 Interprétation des résultats

Si RViz montre un scan LiDAR tourné, une odométrie instable, un repère `base_link` mal orienté ou un robot qui avance virtuellement alors qu'il devrait reculer, cela signifie qu'une correction doit être faite avant tout essai réel. Les causes les plus fréquentes sont :

— une inversion de sens moteur ;

— une mauvaise définition du repère capteur ;

— un paramétrage incorrect du rayon de roue ou de l'entraxe ;

— une erreur de convention sur les vitesses linéaires ou angulaires ;

— une liaison `/tf` incomplète.

Règle de sécurité avant mouvement

Tant que les informations affichées dans RViz ne sont pas cohérentes, le robot ne doit pas être autorisé à se déplacer de manière libre dans la zone d'essai. La validation RViz constitue donc une étape préalable obligatoire avant le démarrage des tests dynamiques.

5.4 Validation de la communication série

La communication série est validée en envoyant des commandes simples depuis la Jetson vers la STM32. Exemple :

Listing 5.1 – Exemple de commande série de test

```
echo "CMD 0.20 0.00 0.00" > /dev/ttyUSB0
echo "STOP" > /dev/ttyUSB0
```

Les réponses attendues sont des trames de télémétrie du type :

Listing 5.2 – Exemple de télémétrie retournée par la STM32

```
HB 1250
ENC 1024 1018 1030 1022
ODOM 0.12 0.00 0.01
```

5.5 Validation de HoloLens et du jumeau numérique AASX Blazor

La validation de l'interface immersive consiste à vérifier que l'utilisateur peut porter le casque HoloLens, ouvrir le dashboard, visualiser le jumeau numérique et commander le robot sans perte d'information importante.

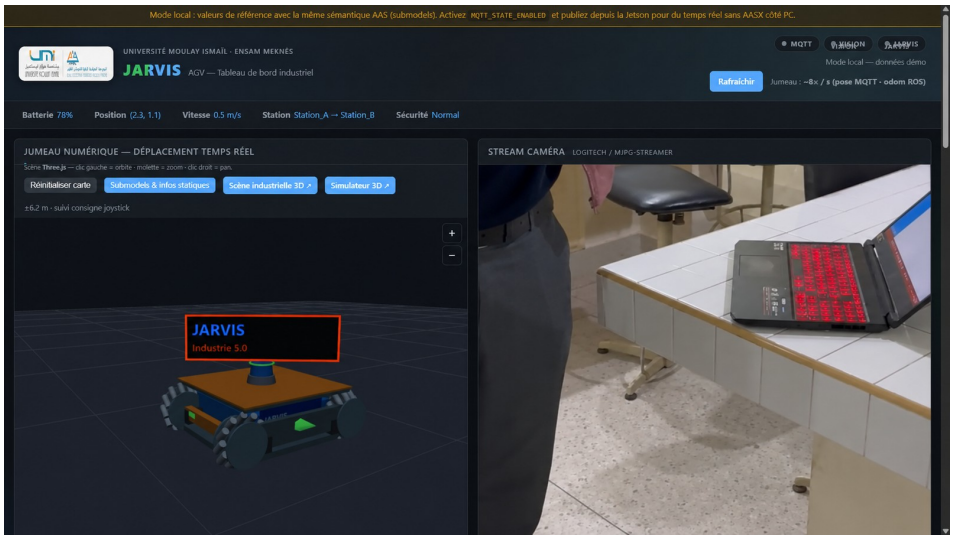


FIGURE 5.1 – Test de l'affichage du dashboard avec jumeau numérique, flux caméra et données de supervision

Les points suivants doivent être contrôlés :

- affichage lisible du dashboard dans le casque ;
- accès aux boutons de commande manuelle et à l'arrêt logiciel ;
- synchronisation de la pose 3D du robot avec la télémétrie MQTT ;
- affichage correct de la scène d'usine, de la trajectoire et des obstacles ;
- basculement clair entre mode manuel, mode mission et mode voice agent ;
- absence de conflit entre les commandes HoloLens, dashboard web et navigation autonome.

5.6 Validation du dashboard

Le dashboard est validé si les fonctions suivantes sont opérationnelles :

— affichage correct de la page web ;

— commande des mouvements ;

— retour de l'état robot ;

— affichage caméra ;

— affichage LiDAR ;

— bouton d'arrêt fonctionnel ;

— indicateurs de connexion visibles.

5.7 Validation des mouvements

Les mouvements de base à vérifier sont :

TABLE 5.3 – Mouvements de base à valider

Mouvement	**Commande attendue**	**Observation**
Avancer	$v_x > 0$, $v_y = 0$, $\omega_z = 0$	Le robot avance droit
Reculer	$v_x < 0$, $v_y = 0$, $\omega_z = 0$	Le robot recule droit
Translation gauche	$v_x = 0$, $v_y > 0$, $\omega_z = 0$	Déplacement latéral
Translation droite	$v_x = 0$, $v_y < 0$, $\omega_z = 0$	Déplacement latéral inverse
Rotation	$v_x = 0$, $v_y = 0$, $\omega_z \neq 0$	Rotation autour du centre
Arrêt	Commande `STOP`	Immobilisation immédiate

5.8 Problèmes rencontrés et actions correctives

TABLE 5.4 – Exemples de problèmes d'intégration et corrections

Problème	**Cause probable**	**Action corrective**
Le robot tourne au lieu d'avancer	Moteur inversé ou roue mal orientée	Inverser le sens moteur ou corriger la matrice
Oscillations moteur	PID mal réglé	Réduire K_p, limiter K_i, ajouter rampe
Perte UART	Mauvais débit ou câble instable	Vérifier baudrate, masse commune et câble
Jetson redémarre	Alimentation insuffisante	Utiliser convertisseur adapté et batterie stable
Image caméra absente	Port occupé ou mauvais périphérique	Vérifier `/dev/video*` et processus actifs
Données LiDAR absentes	Driver ou port incorrect	Vérifier permissions et port série

5.9 Conclusion de validation

Bilan des essais

La validation doit démontrer que le robot peut être commandé depuis le dashboard, le casque HoloLens et le jumeau numérique AASX Blazor, naviguer de manière autonome via Nav2, transmettre ses données de télémétrie et de scan LiDAR depuis le robot réel ou Gazebo, afficher ses capteurs en 2D/3D et exécuter les ordres de déplacement avec un comportement stable et sécurisé.

5.10 Validation du lien voice agent – YOLO

Cette validation vérifie que les détections de la caméra sont réellement exploitables par le voice agent. Le test consiste à placer des objets ou personnes devant le robot, lire les classes détectées par YOLO, puis demander oralement au système ce qu'il voit.

Table 5.5 – Tests voice agent liés à la vision YOLO

Situation	Commande vocale	Résultat attendu
Personne devant le robot	« Qu'est-ce que tu vois ? »	Réponse indiquant une personne détectée
Obstacle devant le robot	« Avance doucement »	Commande refusée ou limitée si la distance est faible
Zone libre	« Lance la mission 1 »	Mission publiée après vérification de sécurité
Détection incertaine	« Va vers l'objet »	Demande reformulée ou commande bloquée

5.11 Validation spécifique du voice agent

Le voice agent est validé en testant des commandes vocales simples, puis des commandes liées aux missions. L'objectif est de vérifier que la phrase prononcée est correctement transformée en intention, puis en commande robot.

Table 5.6 – Tests de validation du voice agent Gemini

Commande vocale	Résultat attendu	Critère de validation
« avance »	Commande avant	Le robot avance à faible vitesse
« arrête-toi »	Commande STOP	Les moteurs s'arrêtent immédiatement
« tourne à gauche »	Rotation gauche	Le robot tourne sur place
« lance la mission 1 »	Publication mission	Le topic `agv/mission` reçoit `mission1`
« donne l'état du robot »	Lecture télémétrie	L'état est affiché ou annoncé

Validation de sécurité

Même si le voice agent fonctionne correctement, l'arrêt d'urgence matériel et logiciel doit rester prioritaire. Une commande vocale ambiguë ne doit pas être exécutée directement.

6. Conclusion générale

Ce travail a permis de concevoir et de documenter une architecture complète de **communication cobotique** pour un AGV intelligent basé sur **Jetson Orin Nano** et **STM32**. L'originalité du projet réside dans la combinaison de plusieurs niveaux techniques au sein d'une même plateforme : commande temps réel, perception, supervision web, jumeau numérique, interaction vocale et représentation normalisée des données par **AAS/AASX**.

Sur le plan matériel, le projet a mis en évidence l'importance d'une séparation claire entre la couche de calcul embarqué et la couche de commande bas niveau. La Jetson assure la coordination globale, les échanges logiciels et les traitements intelligents, alors que la STM32 garantit l'exécution déterministe des commandes moteurs et la lecture fiable des encodeurs.

Sur le plan logiciel, l'intégration de **ROS 2**, **MQTT**, **Node-RED**, **Flask**, **AASX Blazor** et du **voice agent** a permis de construire un environnement cohérent de supervision et d'interaction. L'utilisateur peut observer l'état du robot, le commander manuellement, suivre son déplacement dans une scène 3D et interagir vocalement avec lui. L'ajout de **YOLO** enrichit encore cette interaction en donnant au système la capacité de décrire l'environnement observé par la caméra.

Les essais montrent également l'intérêt de la validation progressive. Avant de mettre le robot en mouvement, la vérification de la chaîne ROS 2 dans **RViz** permet de sécuriser le lancement : cohérence des repères, qualité des scans LiDAR, bonne publication de `/odom`, validité de la pose et stabilité des données capteurs. Cette étape intermédiaire réduit les risques lors des essais réels.

Enfin, le positionnement du projet selon **RAMI 4.0** confirme que la solution couvre une chaîne complète allant de l'actif physique au monde connecté. Le projet constitue donc une base solide pour des développements futurs : navigation autonome plus avancée, flotte multi-robots, cybersécurité, maintenance prédictive, intégration MES/ERP ou encore exploitation industrielle à plus grande échelle.

Bibliographie

[1] ZVEI and Plattform Industrie 4.0, *The Reference Architectural Model Industrie 4.0 (RAMI 4.0)*, Technical report, available online : https://www.plattform-i40.de/.

[2] Plattform Industrie 4.0, *Details of the Asset Administration Shell and RAMI 4.0*, Online resources and working group publications, consulted for architectural positioning.

[3] Industrial Digital Twin Association (IDTA), *Details of the Asset Administration Shell*, Specification series, available online : https://industrialdigitaltwin.org/.

[4] Open Robotics, *ROS 2 Documentation*, Online documentation : https://docs.ros.org/.

[5] Open Robotics, *RViz2 User Guide*, Online documentation : https://docs.ros.org/en/rolling/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html.

[6] Open Navigation LLC, *Navigation2 (Nav2) Documentation*, Online documentation : https://navigation.ros.org/.

[7] Open Robotics, *Gazebo Simulator Documentation*, Online documentation : https://gazebosim.org/docs.

[8] OASIS Standard, *MQTT Version 5.0*, OASIS Committee Specification, 2019.

[9] Ultralytics, *YOLO Documentation*, Online documentation : https://docs.ultralytics.com/.

[10] Microsoft, *ASP.NET Core Blazor Documentation*, Online documentation : https://learn.microsoft.com/aspnet/core/blazor/.

[11] STMicroelectronics, *STM32 Documentation and Application Notes*, Online technical documentation.

[12] NVIDIA, *Jetson Developer Documentation*, Online documentation : https://developer.nvidia.com/embedded/jetson-developer-kit.